FACULTAD DE MEDICINA
UNIVERSIDAD DE CHILE

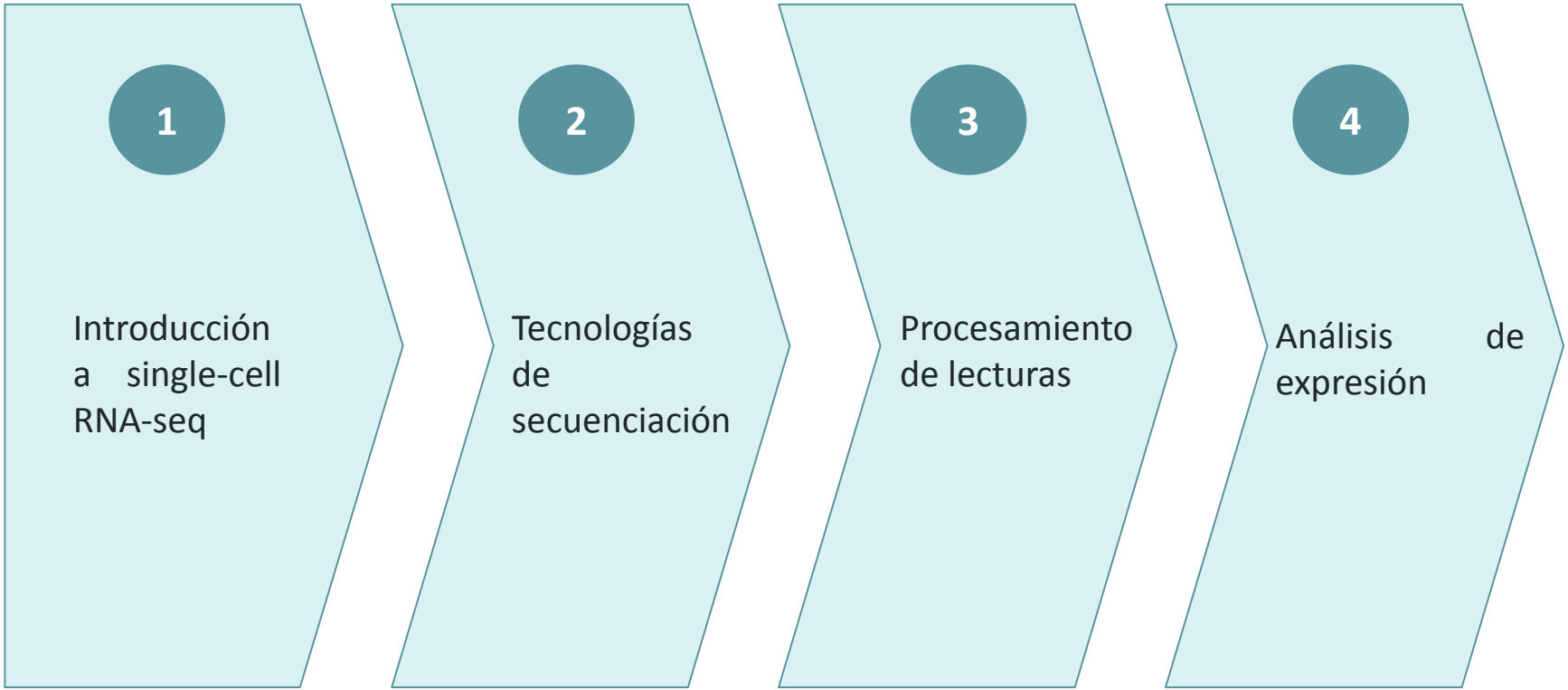
Magíster en Informática Médica

Single-cell RNA-seq I

Noemí Gómez
29 de octubre, 2026

noemi.gomez@postgrado.uoh.cl

Índice de contenidos

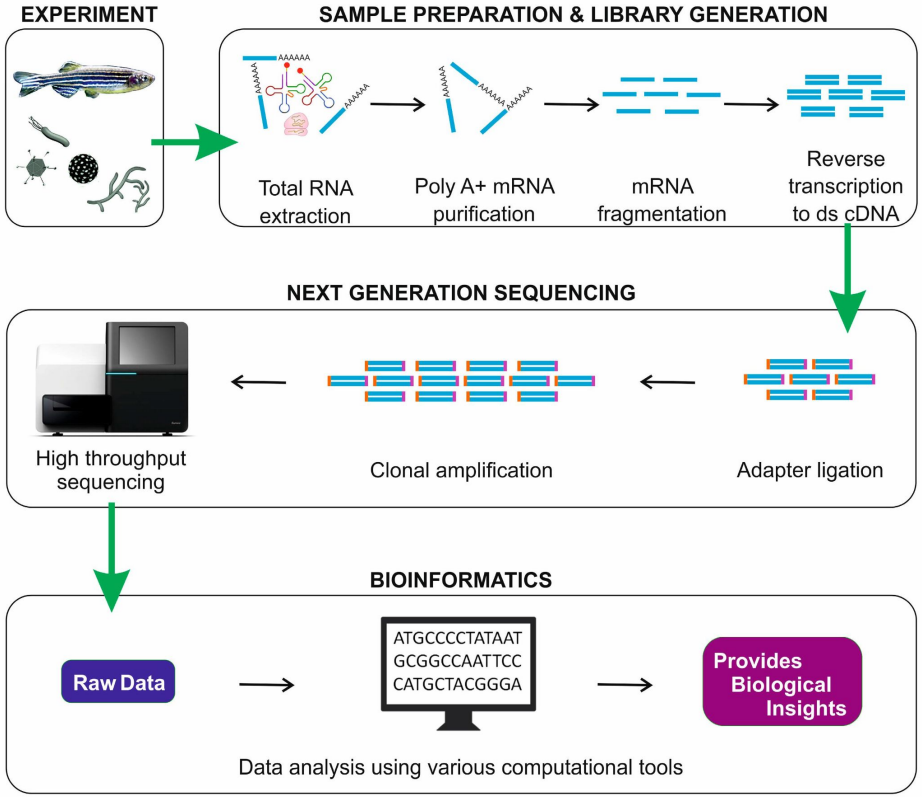


Índice de contenidos

1

Introducción a single-cell RNA-seq

Secuenciación y análisis de RNA

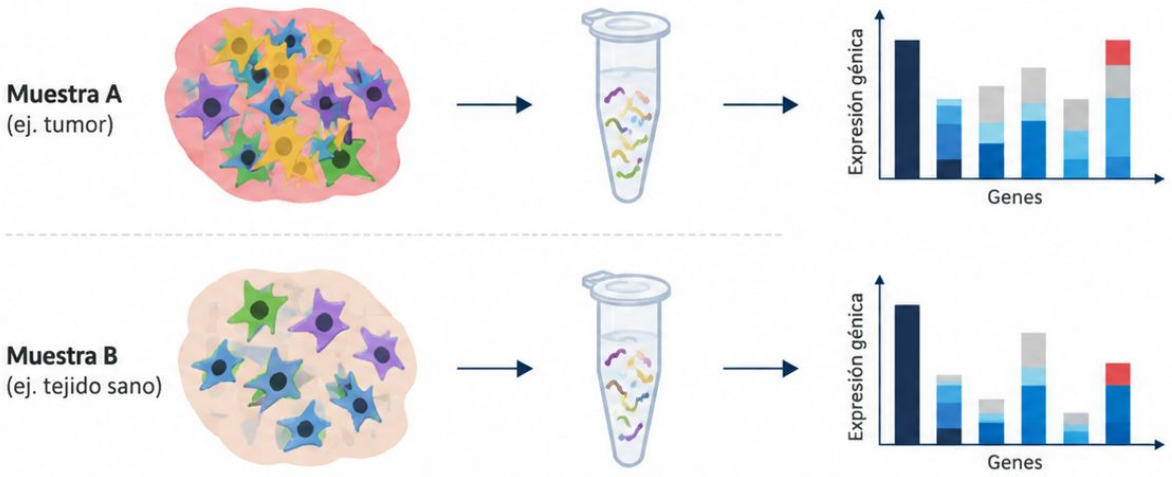


Hasta ahora hemos visto *bulk* RNA-seq…

Muestra/tejido
Compuesto por distintos tipos celulares

Extracción de ARN
de todas las células presentes en la muestra

Perfil de expresión por muestra
Es una señal promedio



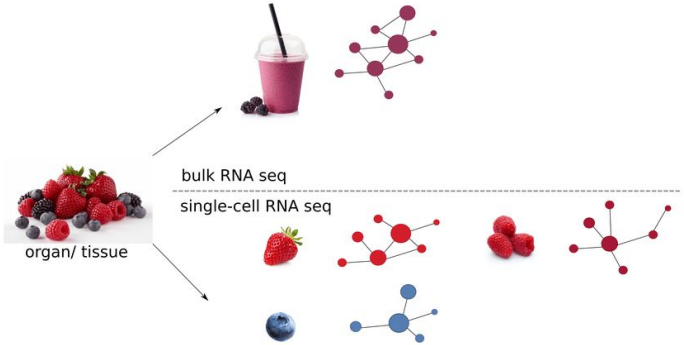
La secuenciación bulk es la secuenciación clásica que representa la expresión promedio de todas las células de una muestra

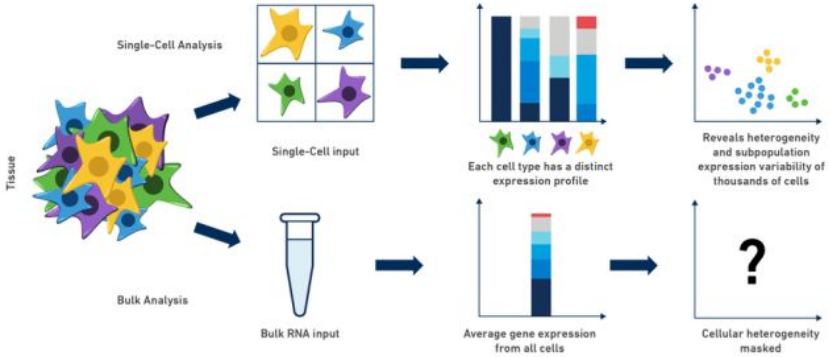
Entonces… ***¿todas las células que componen un tejido tienen el mismo fenotipo y perfil de expresión?***

RNA-seq con resolución de célula única

Qué es
Analiza la expresión génica a nivel de células individuales.

Identidad celular
Cada célula conserva su identidad durante el análisis.





Ventaja
Permite estudiar la heterogeneidad celular y detectar poblaciones distintas.

En lugar de una sola señal promedio, obtenemos múltiples perfiles celulares.

	Bulk	*single-cell*
Cuantificación de la expresión génica	✓	✓
Comparar muestras o condiciones	✓	✓
Identificar tipos/poblaciones celulares	✗	✓
Detectar poblaciones minoritarias	✗	✓
Caracterizar la heterogeneidad celular	✗	✓
Inferir estados y trayectorias celulares	✗	✓
Costo	Bajo	**Alto**
Complejidad	Baja	**Alta**



Bulk RNA-seq

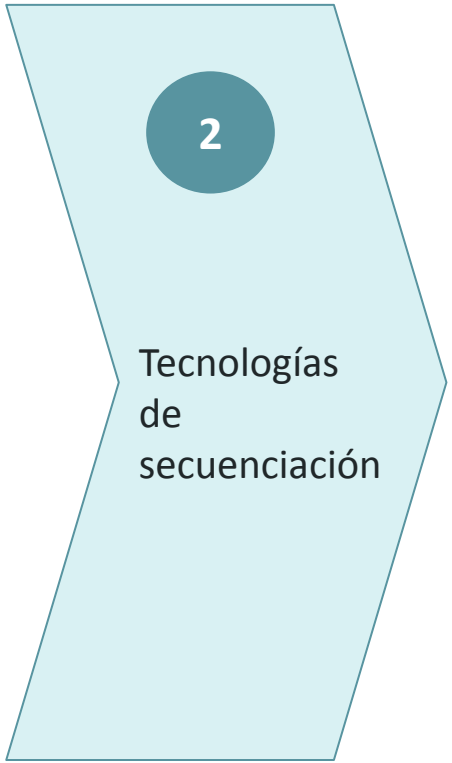
- Expresión promedio de todas las células de la muestra.
- Permite comparar condiciones y cuantificar expresión génica.
- Menor costo y complejidad.



Single-cell RNA-seq

- Expresión génica a nivel de células individuales.
- Permite identificar poblaciones celulares, estados y heterogeneidad.
- Mayor costo y complejidad, con menor profundidad por célula.

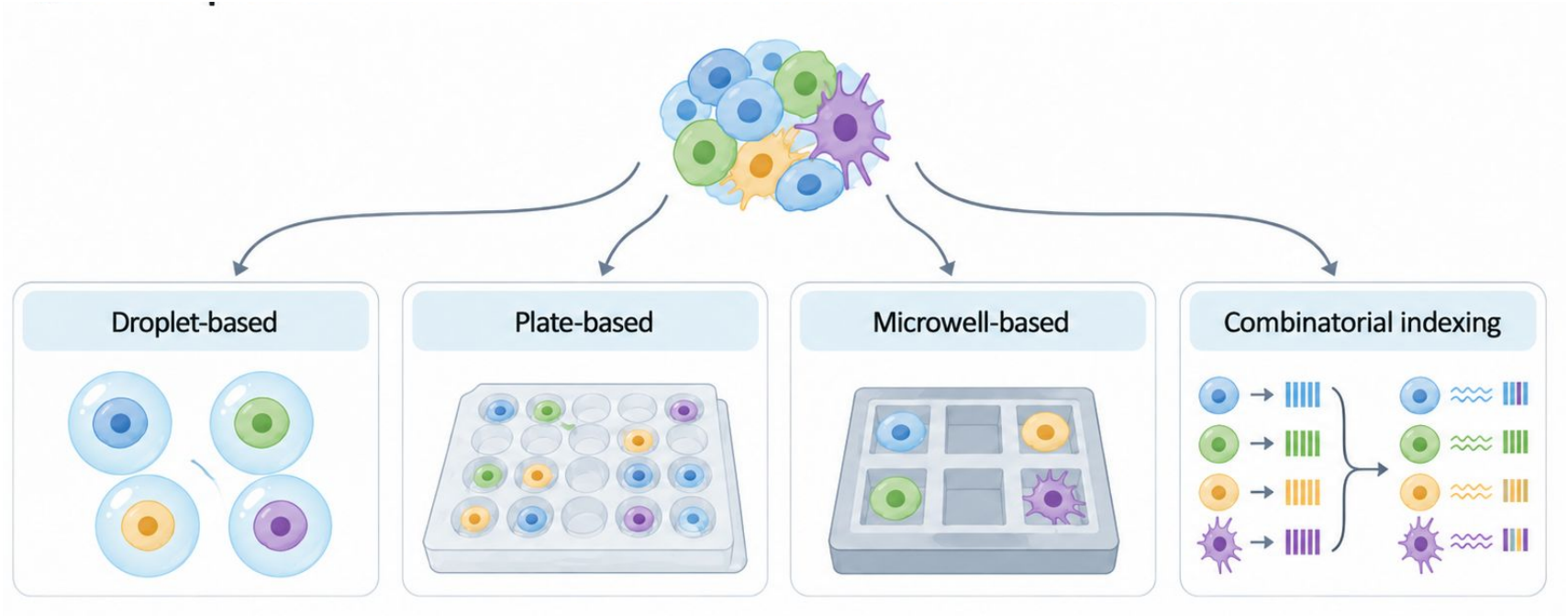
Índice de contenidos



A partir de una muestra se obtiene una suspensión de células individuales

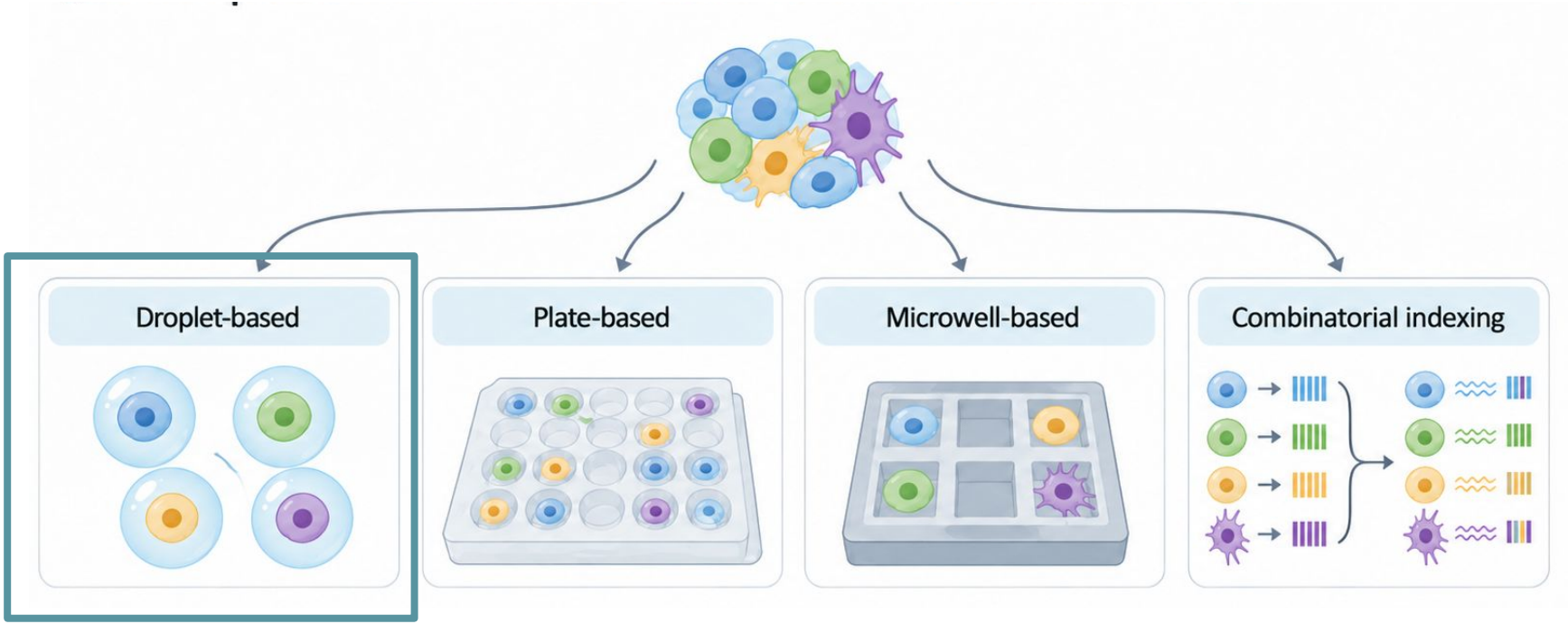


¿Cómo podemos estudiar cada célula individualmente?



Distintas estrategias permiten identificar células individuales previo a secuenciar su RNA.

¿Cómo podemos estudiar cada célula individualmente?



Distintas estrategias permiten identificar células individuales previo a secuenciar su RNA.

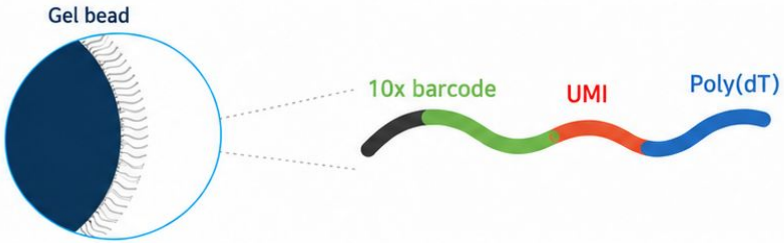
Droplet-based



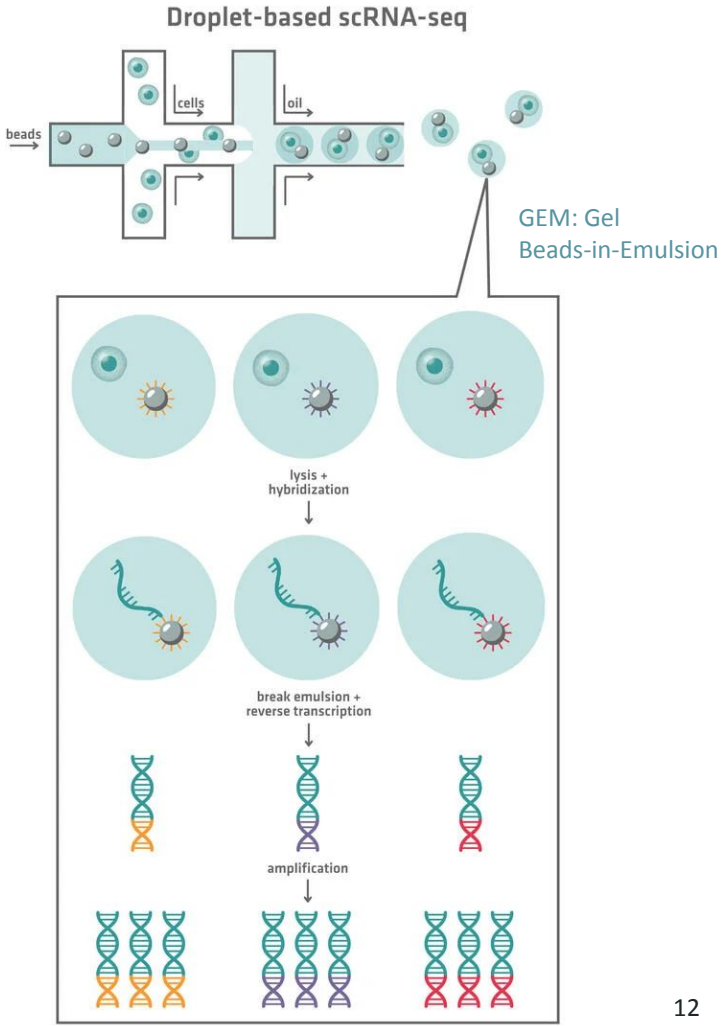
¿Cómo funciona?
Se generan microgotas que contienen aceite, células individuales y *gel beads*



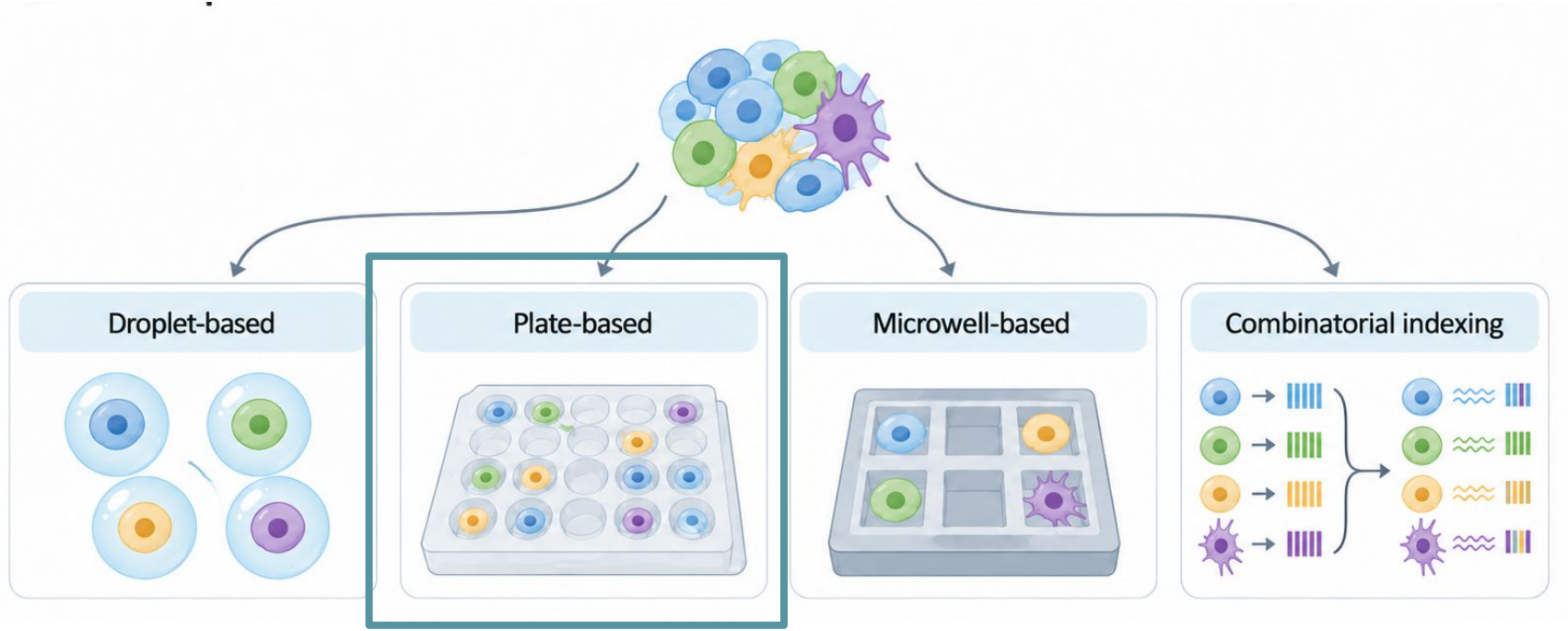
¿Qué aporta el *bead*?
10x barcode: identificador de la célula
UMI: identificador de molécula
poly(dT): captura el RNA celular



(*) El más utilizado es el de 10x CHROMIUM



¿Cómo podemos estudiar cada célula individualmente?

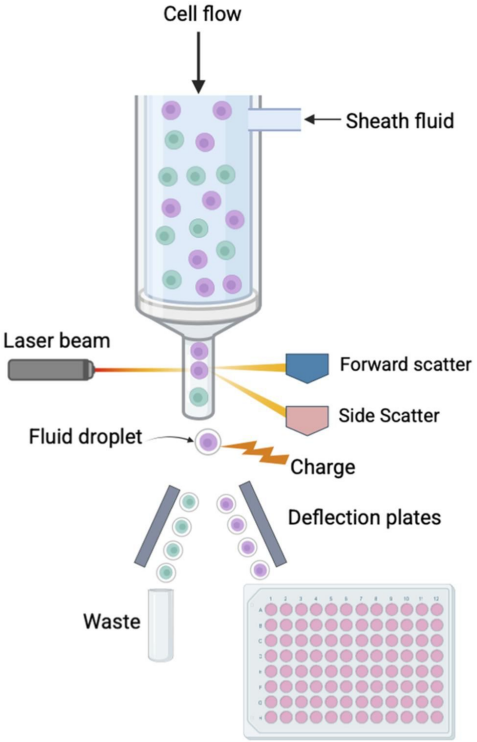


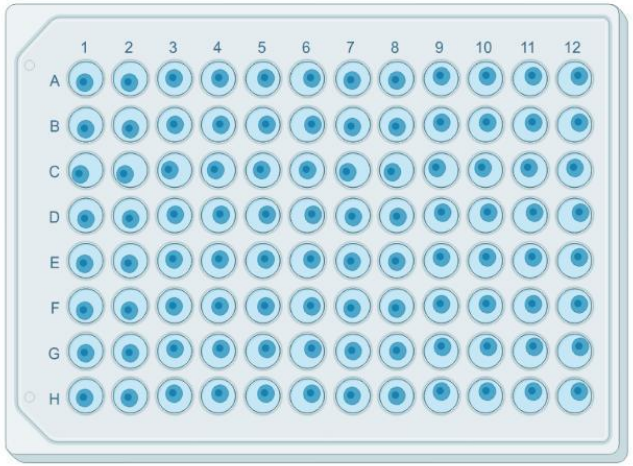
Distintas estrategias permiten identificar células individuales previo a secuenciar su RNA.

Plate-based

FACS
Fluorescence-**A**cti vated Cell Sorting

Las células pasan por un láser y son seleccionadas para ser depositadas en pocillos individuales

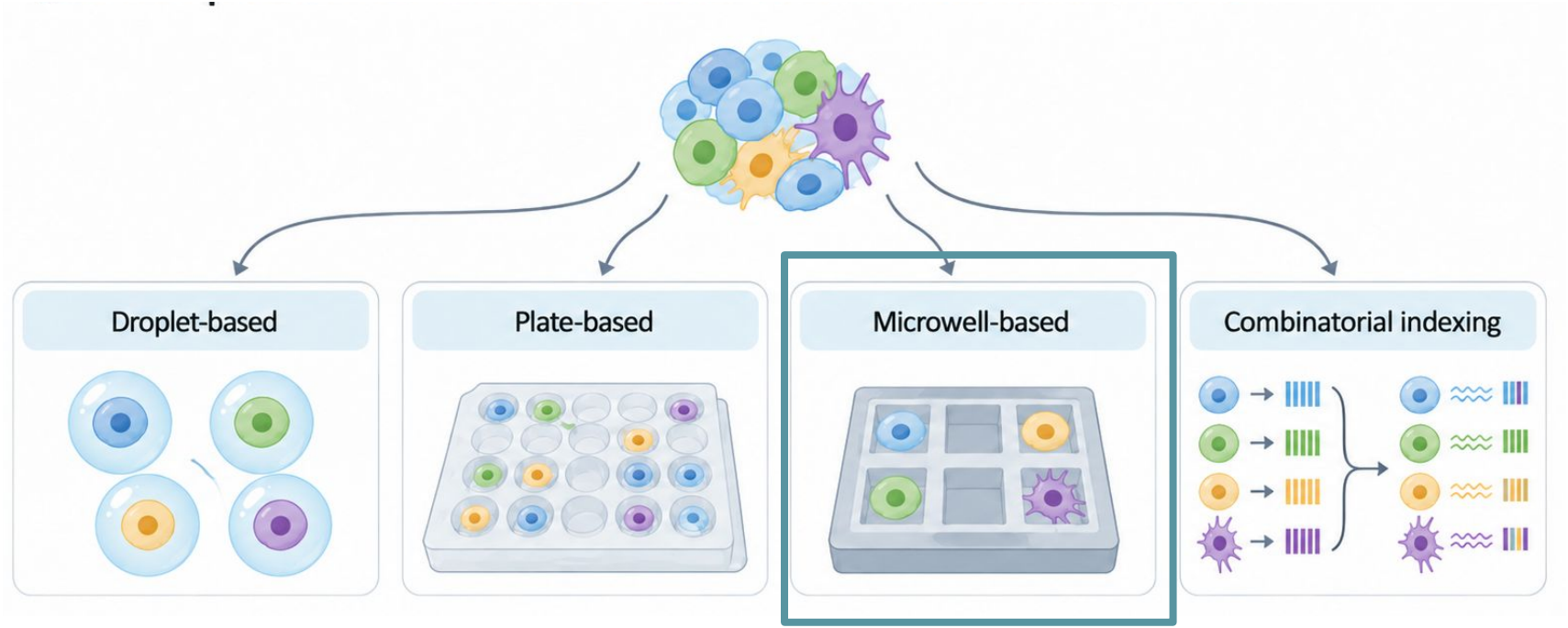




Las bibliotecas para secuenciación se preparan en cada pocillo individualmente, con una célula por pocillo

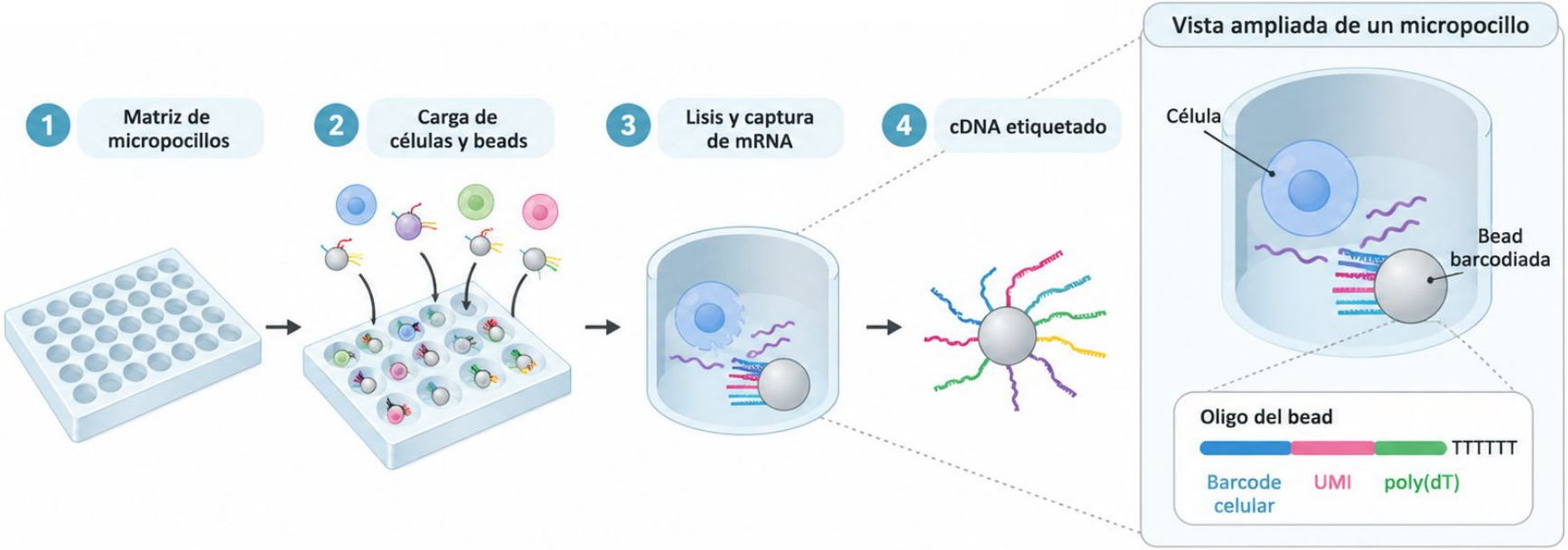
A diferencia de droplet-based, no presentan un barcode celular. La identidad se conserva por el pocillo

¿Cómo podemos estudiar cada célula individualmente?



Distintas estrategias permiten identificar células individuales previo a secuenciar su RNA.

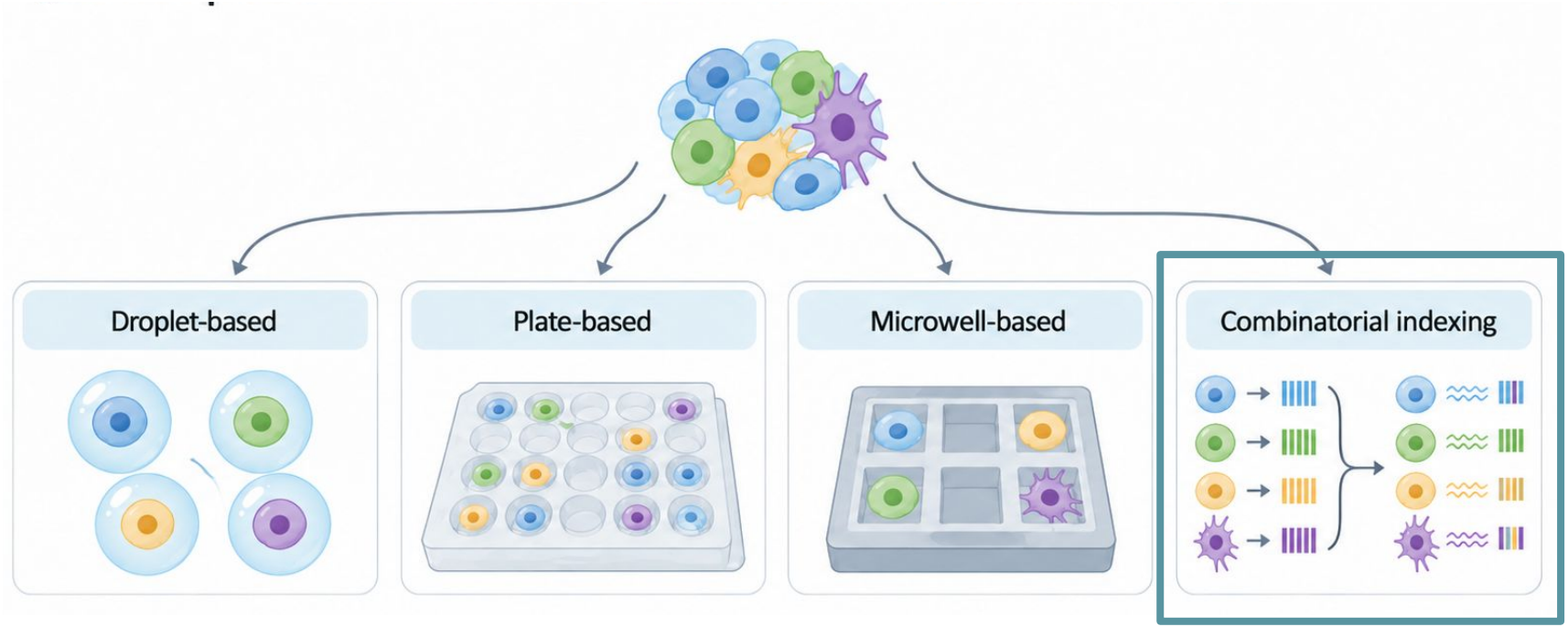
Microwell-based



La técnica basada en micropocillos combina el aislamiento físico de *plate-based* con beads y barcodes de *droplet-based*.

Ejemplo: Seq-Well.

¿Cómo podemos estudiar cada célula individualmente?



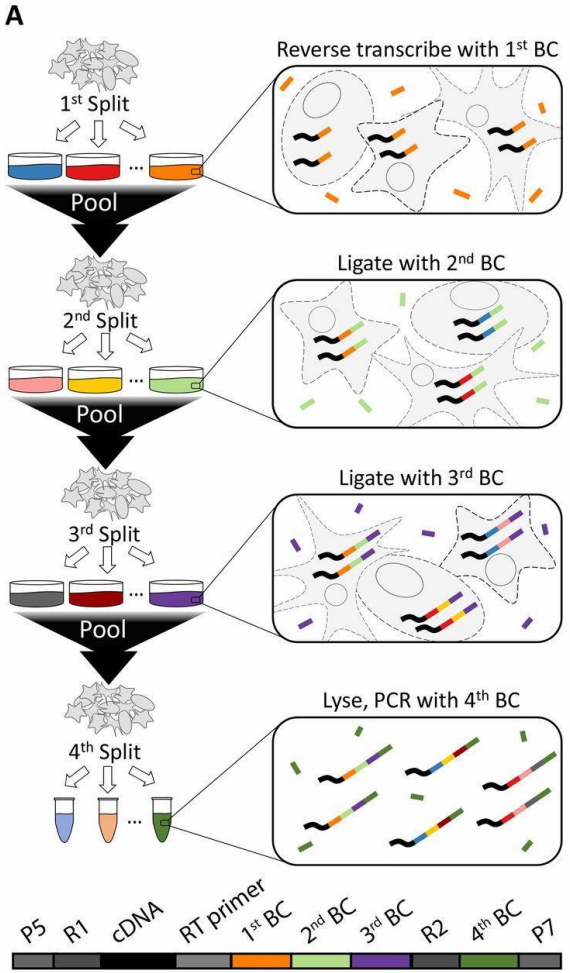
Distintas estrategias permiten identificar células individuales previo a secuenciar su RNA.

Combinatorial indexing

Split-pool indexing
Las células se distribuyen en pocillos, reciben un barcode, se mezclan y se redistribuyen en nuevas rondas.

Cada célula acumula una combinación única de barcodes generadas en cada ronda, lo que permite identificarlas.

Alta escala sin aislamiento individual previo, pero pueden haber colisiones de barcodes y menos sensibilidad por célula.

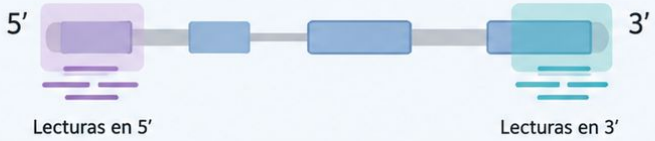


¿Cómo podemos estudiar cada célula individualmente?

	Droplet-based	Plate-based	Microwell-based	Combinatorial indexing
Separación celular	Microgotas (GEMs)	Pocillos individuales	Micropocillos	Rondas de distribución y mezcla
Identidad celular	Barcode de gel bead + UMI	Pocillo + índice de biblioteca	Barcode de bead + UMI	Combinación de barcodes
Cantidad de células	Miles a decenas de miles	Decenas a cientos	Miles	Decenas a cientos de miles
Ejemplos	10x Chromium	Smart-seq2	Seq-Well, BD Rhapsody	SPLiT-seq, sci-RNA-seq
Ventajas	Alta escala	Mayor detalle por célula	Escalable sin microgotas	Muy alta escala
Limitaciones	Doublets, RNA ambiental	Mayor costo por célula	Captura/ocupación variable	Barcode collisions, menor sensibilidad

¿Qué parte del RNA secuenciamos?

Extremos del transcrito (3′ / 5′)



- Captura solo **una región** del transcrito (extremo 3′ o 5′).
- **Estrategia más común en scRNA-seq** (por ejemplo, 10x Chromium).
- Adecuada para **cuantificar expresión génica** en muchas células.
- El extremo capturado puede variar según el protocolo, pero ambos permiten medir expresión por gen.

Transcrito completo (full-length)

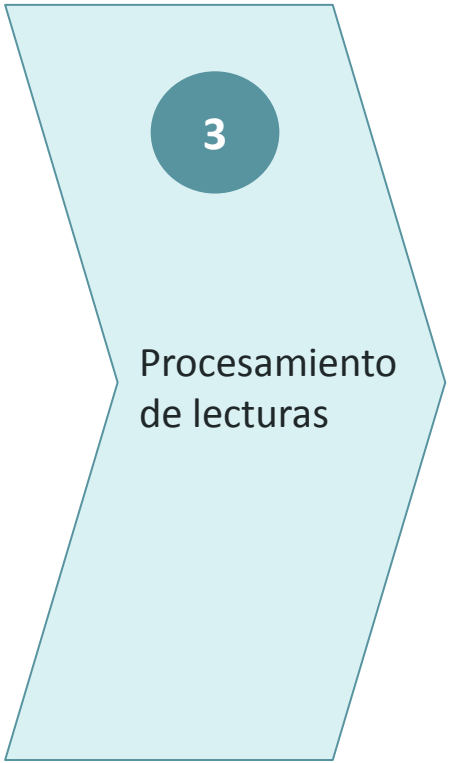


- Captura información a lo largo de todo el transcrito.
- **Permite estudiar isoformas, splicing** y características adicionales del RNA.
- Generalmente se aplica a un menor número de células, con mayor profundidad por célula.
- **Ejemplos**: Smart-seq2, Smart-seq3.

La estrategia de biblioteca se elige según el objetivo del estudio.

- Muchas células / expresión génica → Extremos 3′ / 5′ (más común)
- Más detalle del transcrito (isoformas, splicing) → Full-length

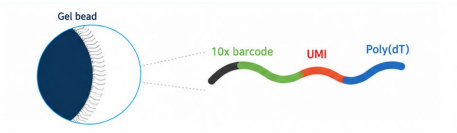
Índice de contenidos



FASTQ

@A00488:61:H7YJCDSX7:1:1101:1532:1000 1:N:0:1	Identificador de lectura
ACGTACGTACGTACGTACGTACGTACGTACGT	***Secuencia***
+	Separador
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	Calidad por base en escala *Phred*

Ejemplo: Lecturas de 10x 3’

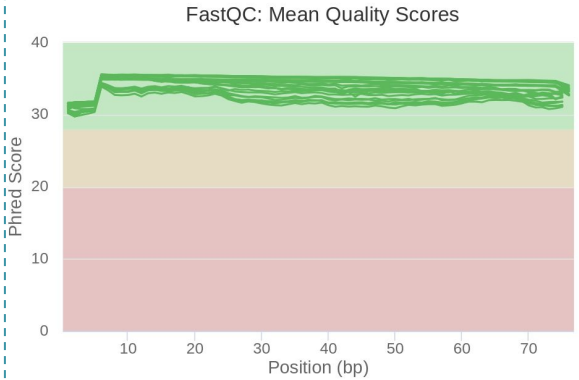


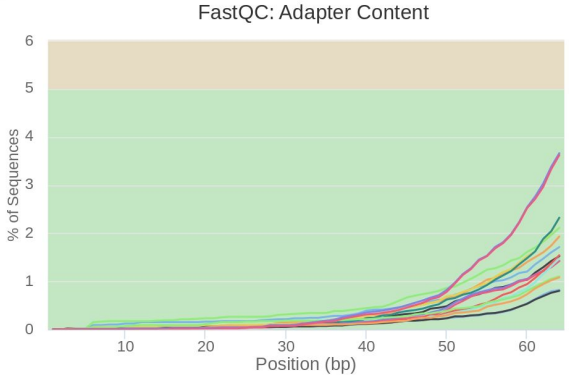
R1	10x barcode (~16 nt)	UMI (~12nt)
R2	cDNA (~50–100 bp)	

La estructura es la misma que para RNAseq, solo que incluye los elementos identificadores

Control de calidad de la secuenciación

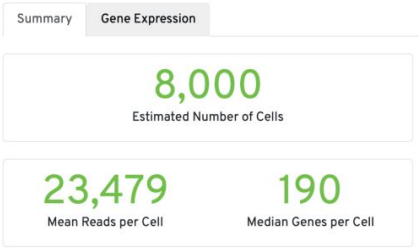
QC habitual de RNA-seq

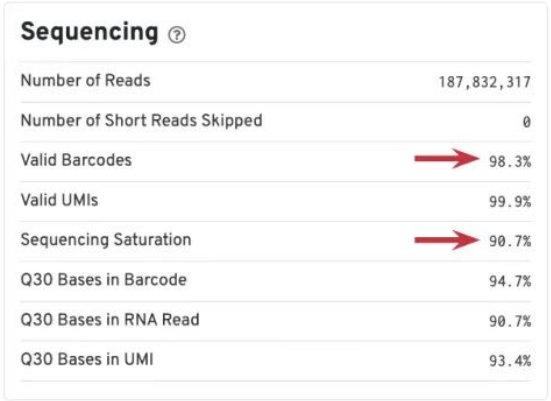




QC específico de scRNA-seq

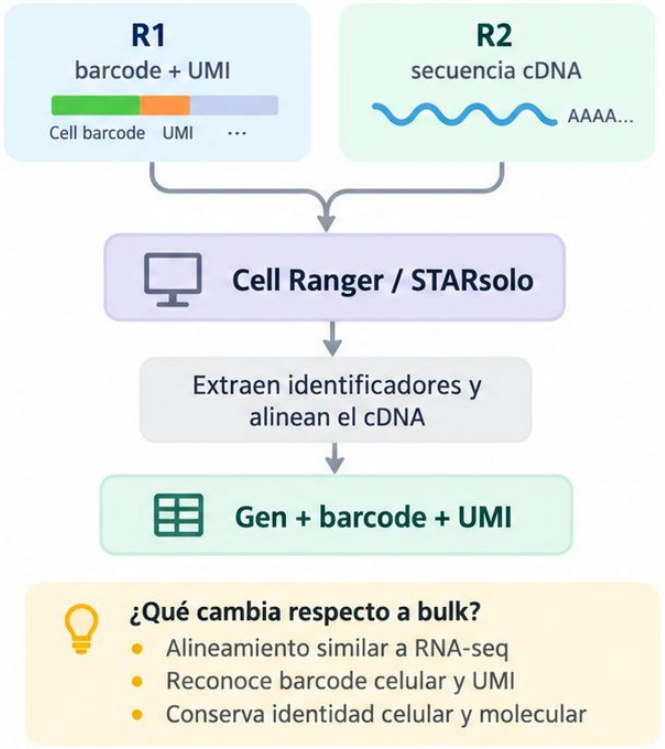
Cell Ranger de 10x permite estimar la cantidad de células de acuerdo a los barcode originales





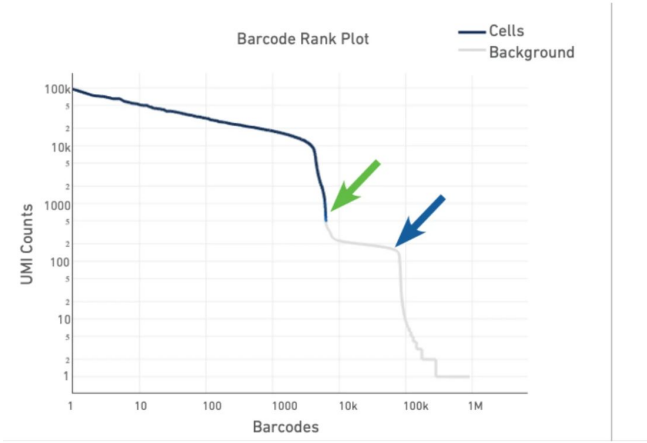
Se revisa la calidad también en las regiones esperadas para la identificación de células

Alineamiento



Mapping

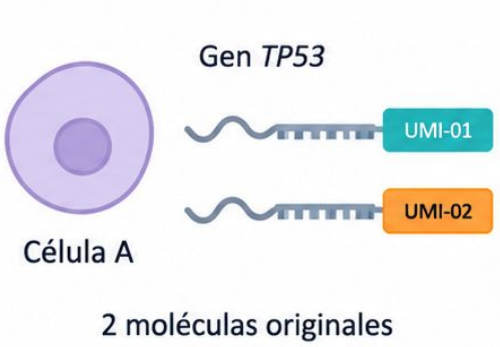
Reads Mapped to Genome	95.2%
Reads Mapped Confidently to Genome	91.3%
Reads Mapped Confidently to Intergenic Regions	5.5%
Reads Mapped Confidently to Intronic Regions	34.6%
Reads Mapped Confidently to Exonic Regions	51.3%
Reads Mapped Confidently to Transcriptome	76.5%
Reads Mapped Antisense to Gene	8.6%



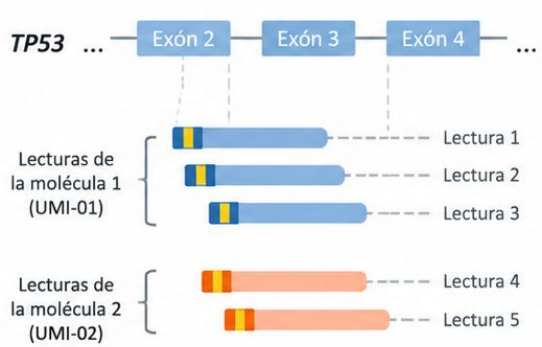
Cell Ranger para 10x 3’
Permite identificar RNA ambiental (barcodes con baja cantidad de UMI)

Cuantificación de la expresión

1. Molecular capturadas



2. Amplificación y secuenciación



5 lecturas alineadas a *TP53*:
3 con UMI-01 y 2 con UMI-02.

3. Conteo por UMI

***TP53* en la célula A**

UMI	Lecturas	Moléculas contabilizadas
UMI-01	3	**1**
UMI-02	2	**1**

Conteo final:
***TP53* = 2 moléculas (UMI)**
en la célula A

Cuantificación = conteo de UMI por gen y célula

Las lecturas alineadas al mismo gen se agrupan por barcode celular y UMI para estimar las moléculas originales capturadas

Matriz célula–gen

Barcodes = **células**

Genes

	BC01	BC02	BC03
TP53	2	0	1
APC	0	4	1
KRAS	3	1	0

En 10x (o tecnologías con UMI) cada valor representa el número de UMI detectados para un gen en una célula.

Asignación de lecturas a cada gen
Utilizando genoma de referencia y anotación.

↓

Agrupación de lecturas de cada gen por barcode y UMI.
Identificación de células y moléculas únicas.

↓

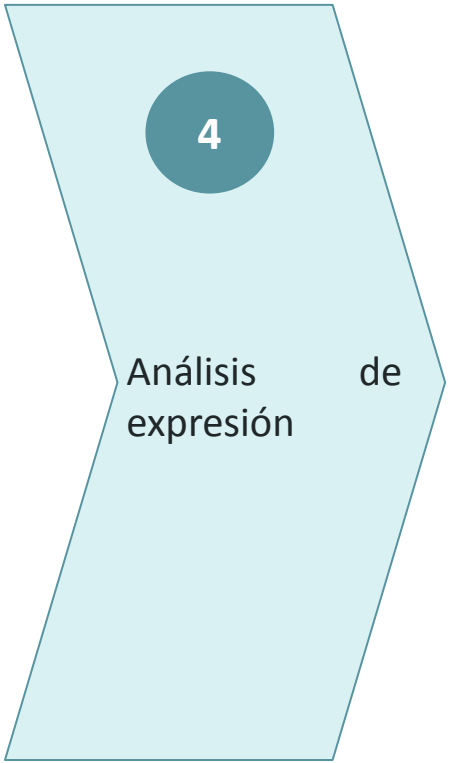
Cuantificación de genes a nivel celular
Por cada célula, ¿cuántos UMI en cada gen?

↓

Cell Calling **permite filtrar las células**
En los droplet-based se puede encontrar RNA ambiental identificados como barcodes con baja cantidad de UMI

Archivos típicos para 10x: matrix.mtx.gz, barcodes.tsv.gz, features.tsv.gz

Índice de contenidos

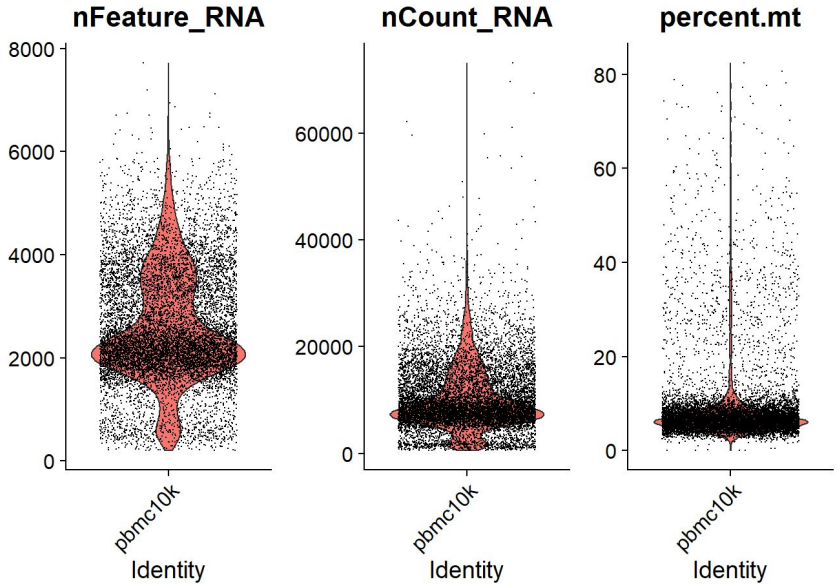


Control de calidad de la cuantificación

nFeature_RNA
Cantidad de genes detectados por célula
Valores bajos: células con poca información.
Valores altos: posible *doublet*

nCount_RNA
Suma de conteos UMI por célula
Valores bajos: pocas moléculas por célula
Valores altos: posible *doublet*

percent.mt
Porcentaje de conteos de genes mitocondriales
Valores altos indican posible estrés o daño celular




```
BiocManager::install("Seurat")

library(Seurat)
pbmc <- CreateSeuratObject(counts = pbmc)
```

```
pbmc[["percent.mt"]] <- PercentageFeatureSet(pbmc, pattern = "^MT-")

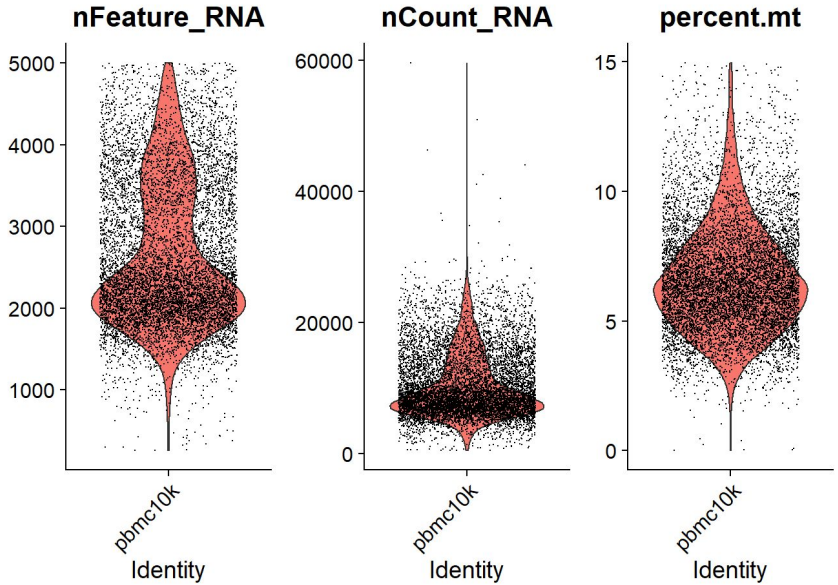
VlnPlot(pbmc, features = c("nFeature_RNA", "nCount_RNA",
"percent.mt"), ncol = 3)
```

Control de calidad de la cuantificación

nFeature_RNA
Cantidad de genes detectados por célula
Valores bajos: células con poca información.
Valores altos: posible *doublet*

nCount_RNA
Suma de conteos UMI por célula
Valores bajos: pocas moléculas por célula
Valores altos: posible *doublet*

percent.mt
Porcentaje de conteos de genes mitocondriales
Valores altos indican posible estrés o daño celular




```
BiocManager::install("Seurat")

library(Seurat)
pbmc <- CreateSeuratObject(counts = pbmc)
```

```
pbmc <- subset(pbmc, subset = nFeature_RNA > 200 &
                              nFeature_RNA < 2500 &
                              percent.mt < 5)
```

Normalización

¿Qué valor nos permitirá identificar y corregir por *profundidad* de secuenciación?

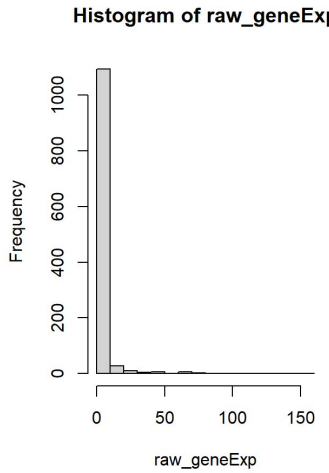
El objetivo de la normalización es reducir las diferencias técnicas entre células

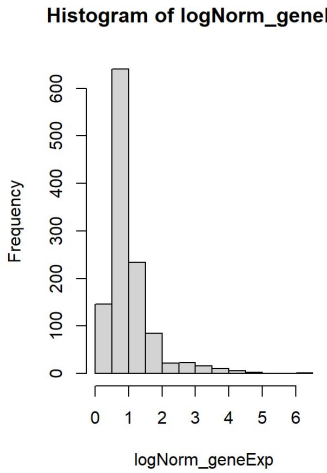
	UMI total	TP53	Proporción	Norm
Célula A	1.000	20	2%	5,3
Célula B	5.000	100	2%	5,3

UMI total por célula = nCount_RNA

$$\text{Expresión normalizada}_{i,j} = \ln\left(1 + \frac{\text{UMI}_{ij}}{\text{UMI totales}_{j}} \times 10\,000\right)$$

```
# Lognormalize de Seurat
pbmc <- NormalizeData(pbmc, normalization.method =
"LogNormalize", scale.factor = 10000)
```





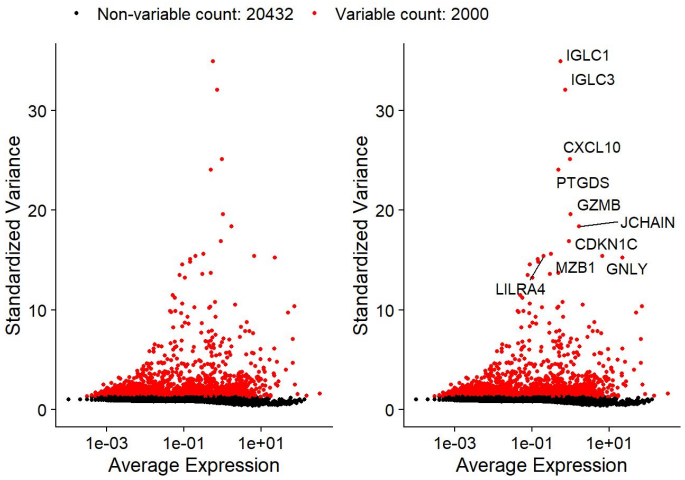
(*) Algunas células si tienen mayor cantidad de ARN

Selección de genes altamente variables (HVG)

Se identifican genes cuya expresión varía considerablemente entre células.
¿Cuánto varía cada gen considerando su expresión promedio?

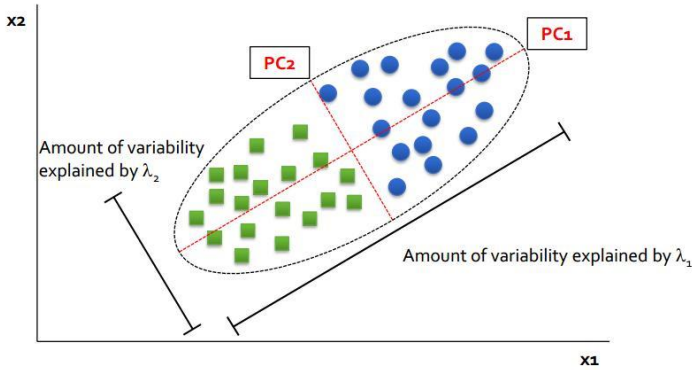
```
pbmc <- FindVariableFeatures (pbmc,
selection.method = "vst", nfeatures = 2000,
verbose = FALSE)
VariableFeaturePlot(pbmc)
```

Permite seleccionar genes informativos y reducir el ruido para análisis posteriores.



¿Cómo resumimos la información de miles de genes?

Análisis de componentes principales (PCA)



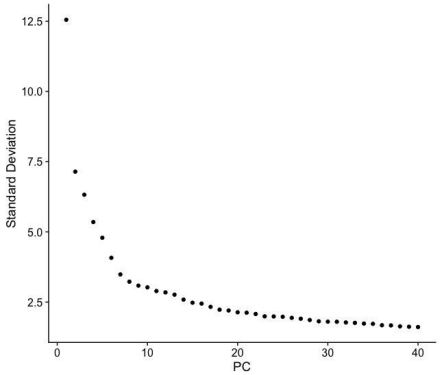
Transforma la expresión de miles de genes en componentes principales que resumen la variación entre celular

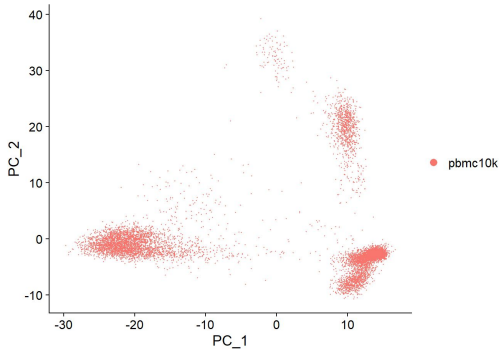
Escalar la expresión

```
all.genes <- rownames(pbmc)
pbmc <- ScaleData(pbmc, features = all.genes)
```

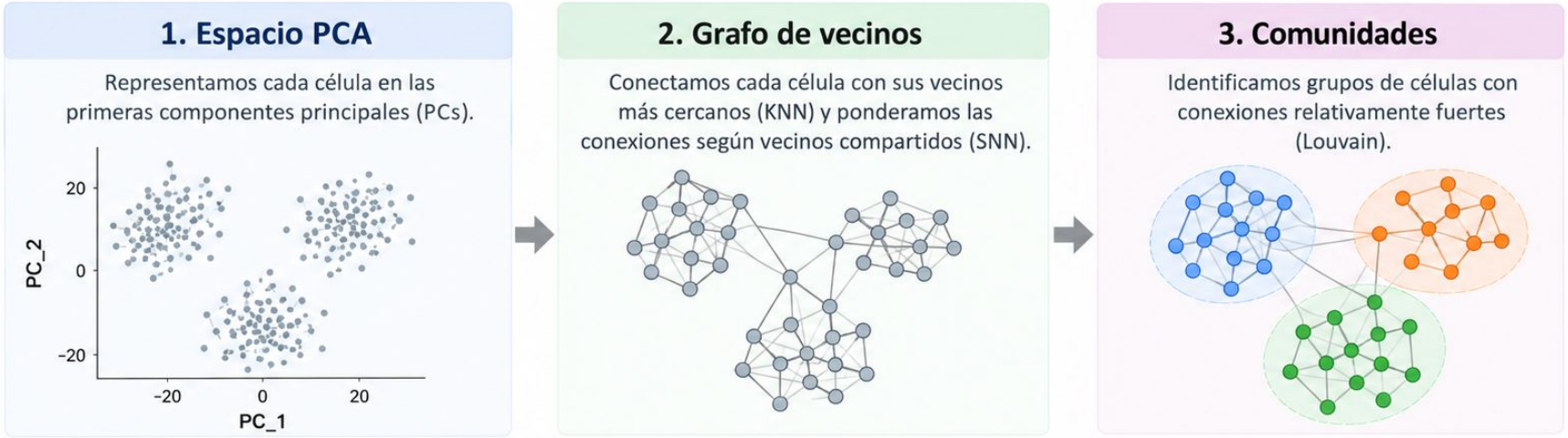
Ejecutar PCA sobre los HVG

```
pbmc <- RunPCA(pbmc, features =
VariableFeatures(object = pbmc))
ElbowPlot(pbmc)
```





Clustering



Construir grafo de vecinos

```
pbmc <- FindNeighbors(pbmc, dims = 1:20)
```

Identificar clusters

```
pbmc <- FindClusters(pbmc, resolution = 0.5)
```

Transforma la expresión de miles de genes en componentes principales que resumen la variación entre celular

Métodos de reducción no lineales

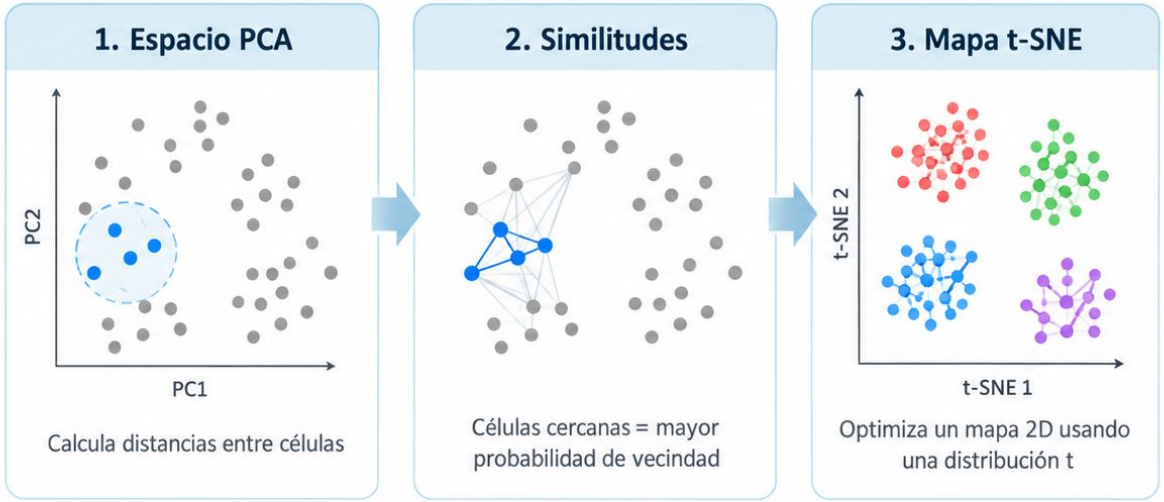
t-SNE: similitud entre células

t-distributed Stochastic Neighbor Embedding

Método no lineal que representa las células similares cerca unas de otras, priorizando conservar relaciones locales

Transforma las distancias entre células en probabilidades de similitud

El nuevo espacio utiliza una distribución t-student para reducir el amontonamiento de puntos, pasando de varias dimensiones a 2D




```
pbmc <- RunTSNE(pbmc, dims = 1:20)
DimPlot(pbmc, reduction = "tsne")
```

Métodos de reducción no lineales

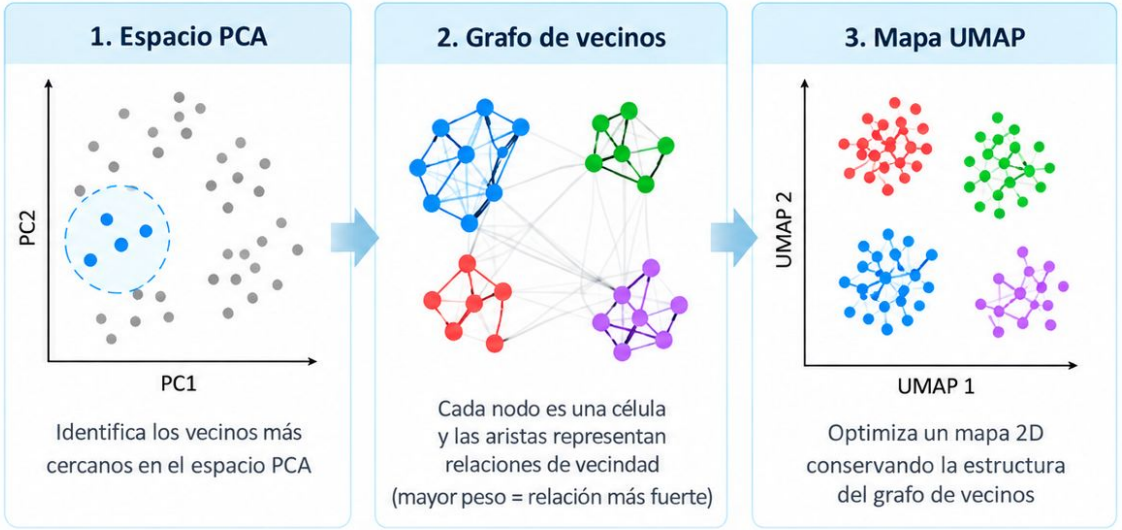
UMAP: relaciones de vecindad entre células

Uniform Manifold Approximation and Projection

Método no lineal que representa las células similares cerca unas de otras, priorizando conservar relaciones locales

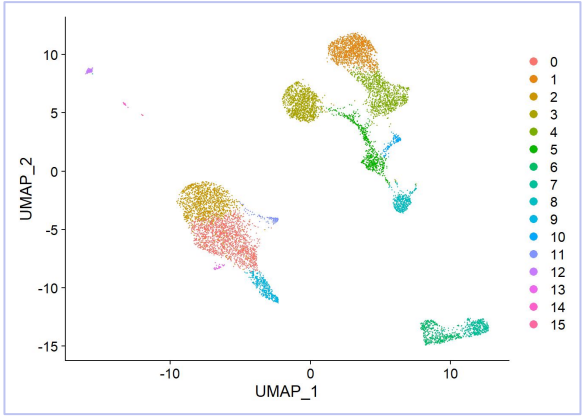
Construye un grafo de vecinos a partir de las distancias entre células

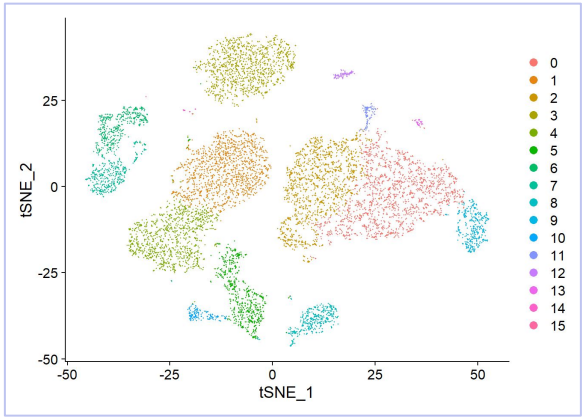
Optimiza el mapa usando parámetro de cantidad de vecinos y distancia mínima



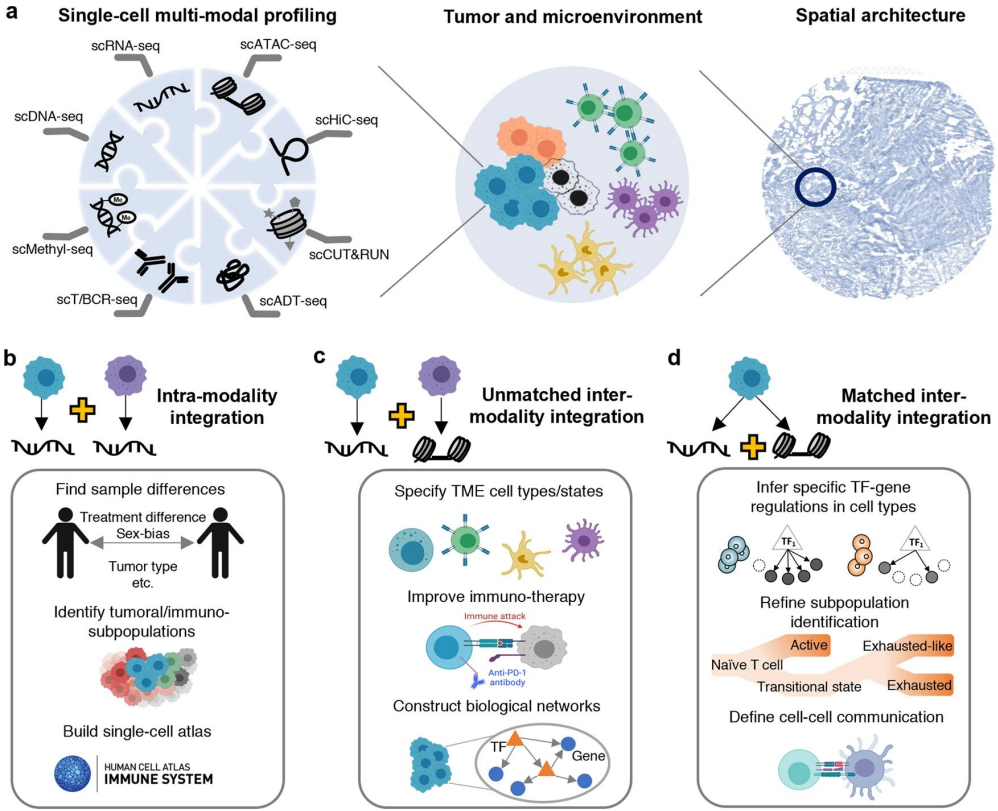

```
pbmc <- RunUMAP(pbmc, dims = 1:20)
DimPlot(pbmc, reduction = "umap")
```

	t-SNE	UMAP
Enfoque	Similitud probabilística	Grafo de relaciones de vecindad
Visualización	Prioriza representar proximidad	Representa relaciones locales y entre regiones
Escalabilidad	Más lento en conjuntos grandes	Más rápido y escalable
¿Cuando utilizar?	Exploración de similitud	Exploración general de la heterogeneidad y organización celular





No solo se secuencia RNA a nivel de célula única



Taller práctico

Explorar matriz de conteo de scRNA-seq de cerebro embrionario de ratón utilizando el paquete Seurat en R:

- De acuerdo a las dimensiones, ¿cuántos genes se están evaluando?
- Evaluar la calidad de la cuantificación y filtrar la matrices con los parámetros vistos en clase *(Realizar gráficos antes y después por separado y de relación entre los valores).* ¿Cuántos genes y células son eliminadas?
- Normalizar la matriz y comparar antes y después, ¿como es la distribución?
- Explorar un PCA con los datos determinando la cantidad óptima de PC para continuar. ¿Qué genes están *moviendo* los componentes?
- Identificar clusters de los datos. ¿Cuántos clusters se generan y cuantas células los componen?
- Ejecutar UMAP y tSNE con los datos de PCA y clustering. Visualiza y compara. Usa parámetros por defecto y también cambia los valores para analizar las diferencias.
    - tSNE. Parámetro perplexity = 30 (defecto). Revisar 50 y 10.
    - UMAP. Parámetros n.neighbors = 30 y min.dist = 0.3. Probar 50, 0.5, y 10 y 0.1.

Opcional

Repetir PCA-Clustering-UMAP/tSNE sin normalizar, ¿cómo cambian los resultados la normalización?

Repite el flujo de análisis sin Seurat. ¿Se replican los resultados?

https://github.com/noemigomezr/scRNAseq_MIM/

Instalar paquetes y descargar datos

```
BiocManager::install(“SingleCellExperiment”)
BiocManager::install(“Seurat”)
BiocManager::(“TENxBrainData”)

library(tidyverse)
library(SingleCellExperiment)
library(Seurat)
library(TENxBrainData)
```

```
sce <- TENxBrainData20k()
set.seed(123)
sce <- sce[, sample(ncol(sce), 3000)] # 3000 cantidad de células

simbolos <- as.character(rowData(sce)$Symbol)
ensembl <- as.character(rowData(sce)$Ensembl)
simbolos[is.na(simbolos) | simbolos == ""] <-
  ensembl[is.na(simbolos) | simbolos == ""]
rownames(sce) <- make.unique(simbolos)
```

```
# Asignar identificadores celulares únicos
colnames(sce) <- make.unique(
  paste(colData(sce)$Library, colData(sce)$Barcode, sep = "_")
)

# Extraer matriz de UMI y crear objeto Seurat
counts <- as(assay(sce, "counts"), "dgCMatrix")
obj <- CreateSeuratObject(counts = counts, project = "Brain_E18")

obj
```

Objeto de SingleCellExperiment

sce	S4 [27998 x 3000] (SingleCell	S4 object of class SingleCellExperiment
int_elementMetadata	S4 [27998 x 1] (S4Vectors::DF	S4 object of class DFrame
int_colData	S4 [3000 x 3] (S4Vectors::DFr	S4 object of class DFrame
int_metadata	list [1]	List of length 1
rowRanges	S4 (GenomicRanges::Compres	S4 object of class CompressedGRangesList
colData	S4 [3000 x 4] (S4Vectors::DFr	S4 object of class DFrame
assays	S4 [27998 x 3000] (Summariz	S4 object of class SimpleAssays
NAMES	NULL	Pairlist of length 0
elementMetadata	S4 [27998 x 0] (S4Vectors::DF	S4 object of class DFrame
metadata	list [0]	List of length 0

Objeto de Seurat

obj	S4 [27998 x 3000] (SeuratObj	S4 object of class Seurat
assays	list [1]	List of length 1
meta.data	list [3000 x 3] (S3: data.frame	A data.frame with 3000 rows and 3 columns
active.assay	character [1]	'RNA'
active.ident	factor	Factor with 1 level: "Brain_E18"
graphs	list [0]	List of length 0
neighbors	list [0]	List of length 0
reductions	list [0]	List of length 0
images	list [0]	List of length 0
project.name	character [1]	'Brain_E18'
misc	list [0]	List of length 0
version	list [1] (S3: package_version,	List of length 1
commands	list [0]	List of length 0
tools	list [0]	List of length 0

Explorar la matriz y objeto Seurat

```
cat("Células:", ncol(obj), "\nGenes:", nrow(obj), "\n")
```

```
Células: 3000
Genes: 27998
```

```
head(rownames(obj))
head(colnames(obj))
assayNames(sce)
```

```
[1] "Xkr4"    "Gm1992"  "Gm37381" "Rp1"     "Rp1.1"
"Sox17"
[1] "126 GGCGACTTCTTGCATT-126" "64_AAGTCTGTCCTCATTA-64"
"10 ACACCAACACGGTTTA-10"
[4] "104_TGCCAAAGTGCCTGTG-104" "60_TACGGGCGTCGGGTCT-60"
"70_TGCCCTATCAAACAAG-70"
[1] "counts"
```

```
as.matrix(counts[1:5, 1:5])
```

```
       126_GGCGACTTCTTGCATT-126 64_AAGTCTGTCCTCATTA-64
10_ACACCAACACGGTTTA-10 104_TGCCAAAGTGCCTGTG-104
Xkr4                             0                        0
0                        0
Gm1992                           0                        0
0                        0
Gm37381                          0                        0
0                        0
Rp1                              0                        0
0                        0
Rp1.1                            0                        0
0                        0
        60_TACGGGCGTCGGGTCT-60
Xkr4                         0
Gm1992                       0
Gm37381                      0
Rp1                          0
Rp1.1                        0
```

Evaluar QC

```
genes_mt <- grep("^mt-", rownames(obj), value = TRUE)
genes_mt
```

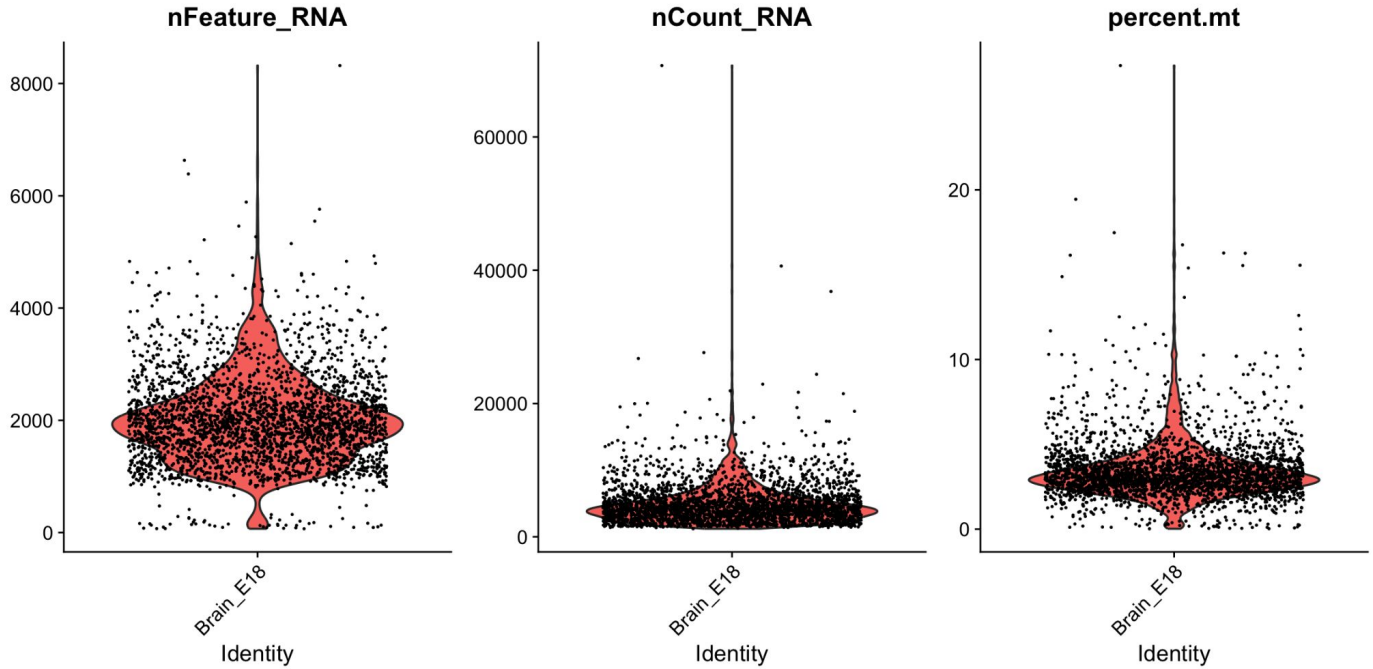
```
[1] "mt-Nd1"  "mt-Nd2"  "mt-Co1"  "mt-Co2"  "mt-Atp8" "mt-Atp6" "mt-Co3"  "mt-Nd3"  "mt-Nd4l" "mt-Nd4"
[11] "mt-Nd5"  "mt-Nd6"  "mt-Cytb"
```

```
obj[["percent.mt"]] <- PercentageFeatureSet(obj, features = genes_mt)
metricas_qc <- c("nFeature_RNA", "nCount_RNA", "percent.mt")
summary(obj[[]][, metricas_qc])
```

```
 nFeature_RNA    nCount_RNA      percent.mt
Min.   :  68   Min.   : 1210   Min.   : 0.02087
1st Qu.:1480   1st Qu.: 2990   1st Qu.: 2.49046
Median :1916   Median : 4216   Median : 3.09585
Mean   :1996   Mean   : 4888   Mean   : 3.42702
3rd Qu.:2362   3rd Qu.: 5792   3rd Qu.: 3.88937
Max.   :8320   Max.   :70710   Max.   :27.32459
```

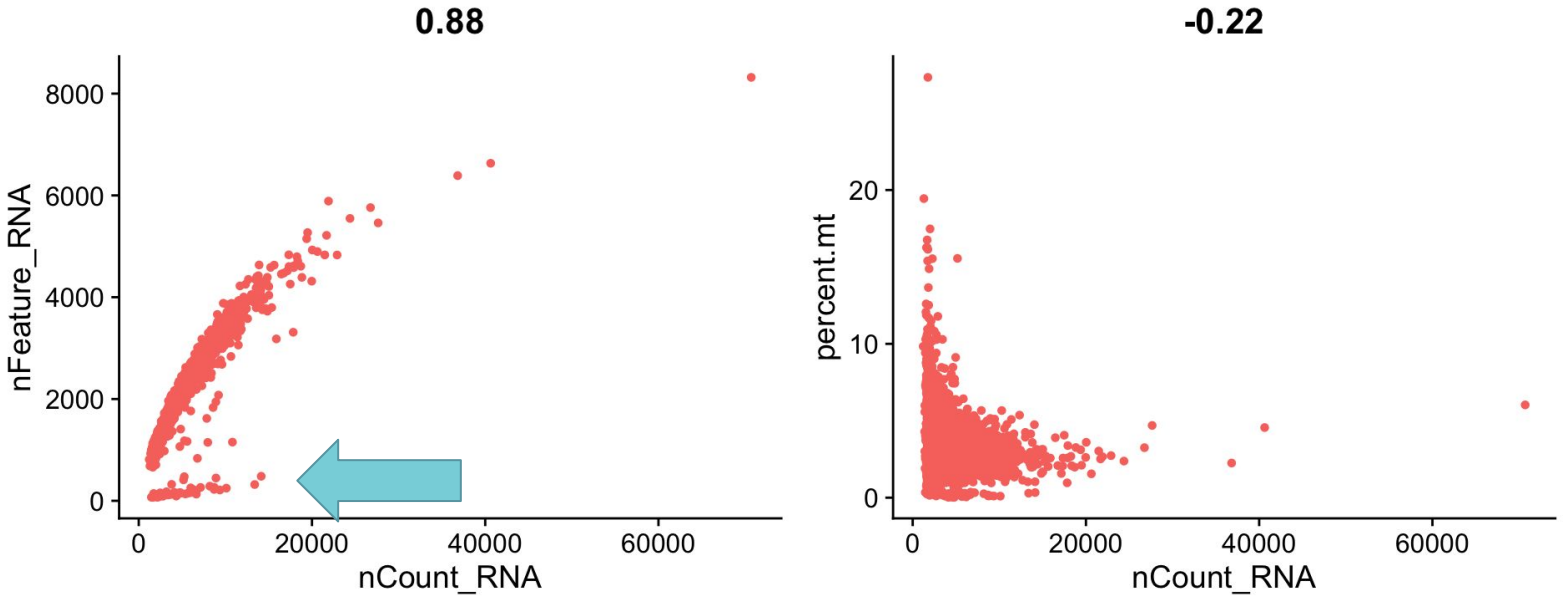
Evaluar QC

```
VlnPlot(obj, features = metricas_qc, layer = "counts", ncol = 3, pt.size = 0.1) + NoLegend()
```



Evaluar QC

```
plot1 <- FeatureScatter(obj, feature1 = "nCount_RNA", feature2 = "nFeature_RNA") + NoLegend()
plot2 <- FeatureScatter(obj, feature1 = "nCount_RNA", feature2 = "percent.mt") + NoLegend()
plot1 + plot2
```



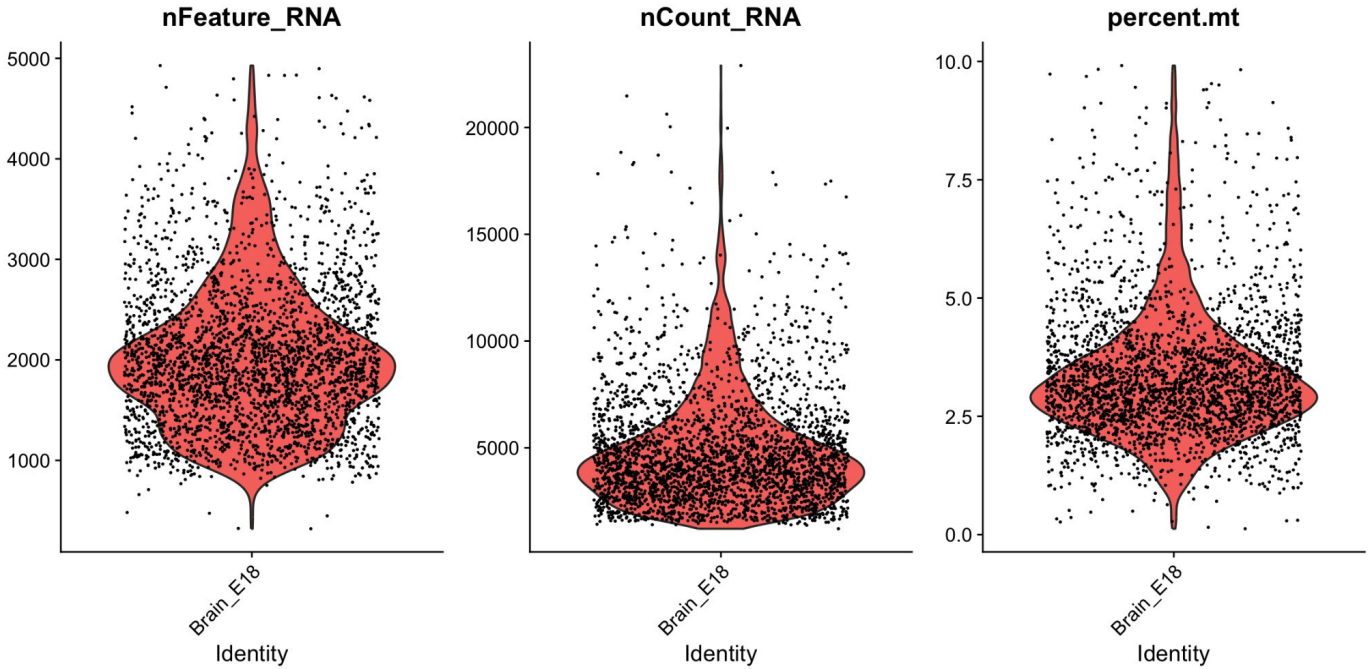
Filtrar por control de calidad

```
antes <- c(Celulas = ncol(obj), Genes = nrow(obj),
           Genes_detectados = sum(Matrix::rowSums(counts) > 0))
obj <- subset(obj, subset = nFeature_RNA > 300 & nFeature_RNA < 5000 & percent.mt < 10)
despues <- c(Celulas = ncol(obj), Genes = nrow(obj),
             Genes_detectados = sum(Matrix::rowSums(GetAssayData(obj, layer = "counts")) > 0))
data.frame(Antes = antes, Despues = despues, Eliminados = antes - despues,
           Perdida_pct = round(100 * (antes - despues) / antes, 2))
```

```
                Antes Despues Eliminados Perdida_pct
Celulas          3000    2911         89        2.97
Genes           27998   27998          0        0.00
Genes_detectados 17485   17417         68        0.39
```

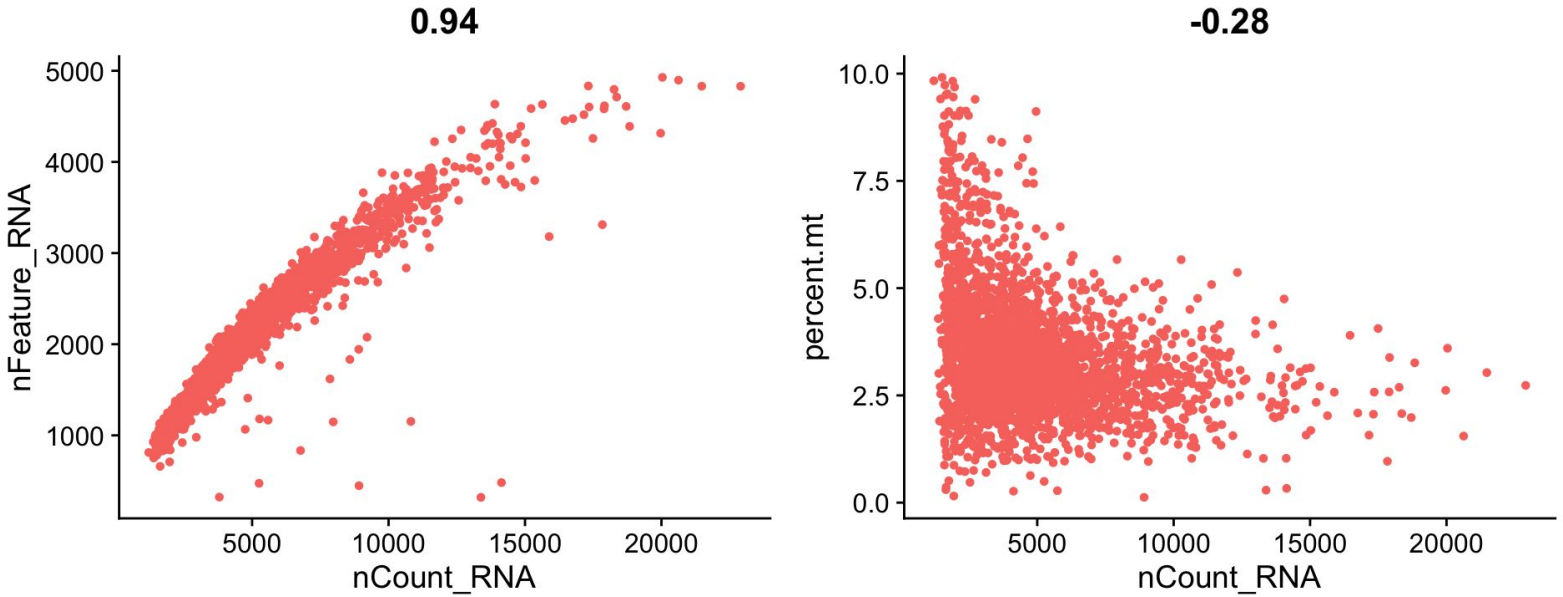
Filtrar por control de calidad

```
VlnPlot(obj, features = metricas_qc, layer = "counts", ncol = 3, pt.size = 0.1) + NoLegend()
```



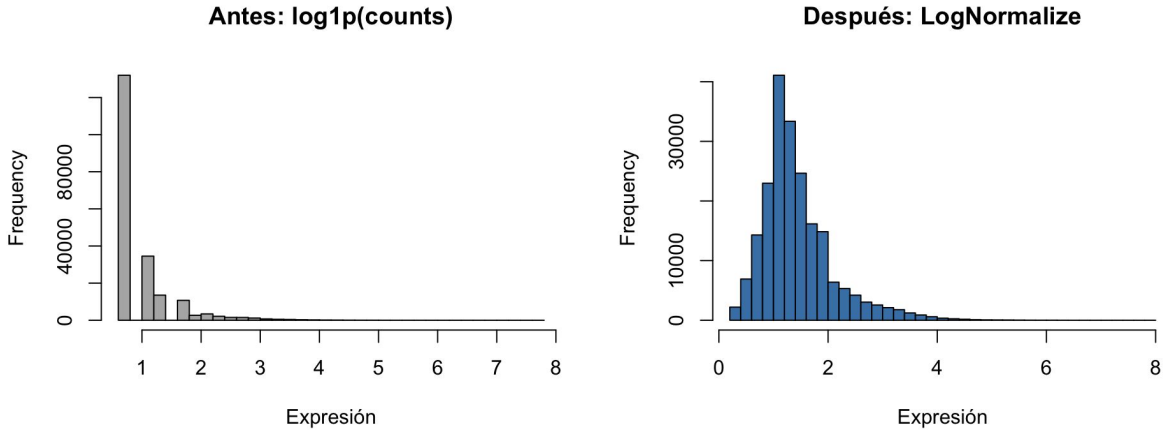
Filtrar por control de calidad

```
plot1 <- FeatureScatter(obj, feature1 = "nCount_RNA", feature2 = "nFeature_RNA") + NoLegend()
plot2 <- FeatureScatter(obj, feature1 = "nCount_RNA", feature2 = "percent.mt") + NoLegend()
plot1 + plot2
```



Normalización

```
obj <- NormalizeData(obj, verbose = FALSE)
celdas <- head(colnames(obj), 100)
originales <- GetAssayData(obj, layer = "counts")[, celdas]
normalizados <- GetAssayData(obj, layer = "data")[, celdas]
par(mfrow = c(1, 2))
hist(log1p(originales@x), breaks = 50, col = "grey70", main = "Antes: log1p(counts)", xlab = "Expresión")
hist(normalizados@x, breaks = 50, col = "steelblue", main = "Después: LogNormalize", xlab = "Expresión")
```

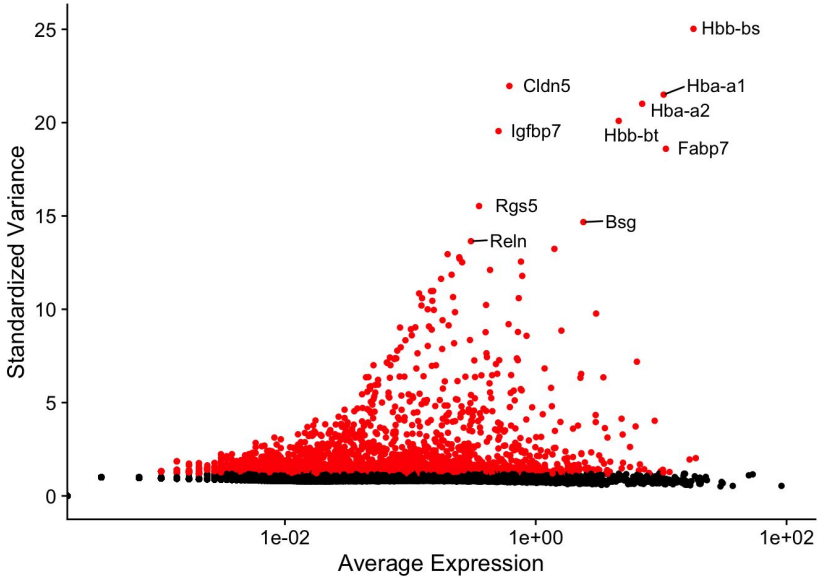


PCA

```
# Encontrar los genes más variables para seleccionar
obj <- FindVariableFeatures(obj, selection.method = "vst", nfeatures = 2000, verbose = FALSE)
top10 <- head(VariableFeatures(obj), 10)
top10
LabelPoints(plot = VariableFeaturePlot(obj), points = top10, repel = TRUE) + NoLegend()
```

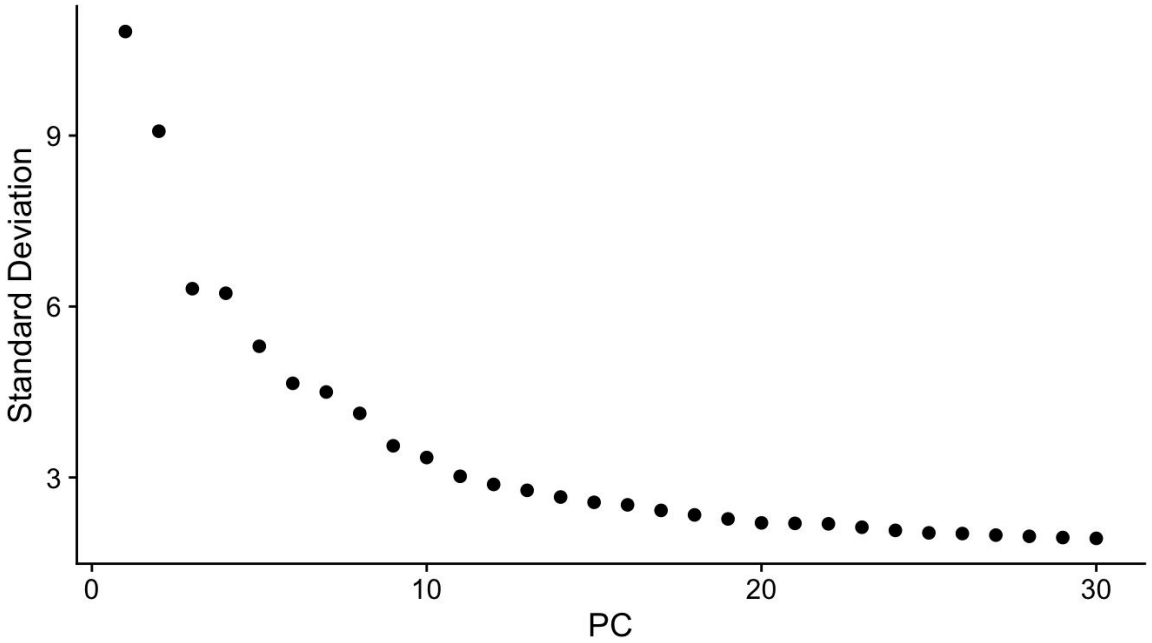
```
[1] "Hbb-bs" "Cldn5"  "Hba-a1" "Hba-a2"
"Hbb-bt" "Igfbp7" "Fabp7"  "Rgs5"   "Bsg"
"Reln"
```

```
# Escalar y correr PCA
obj <- ScaleData(obj,
                 features =
VariableFeatures(obj),
                 verbose = FALSE)
obj <- RunPCA(obj,
              features = VariableFeatures(obj),
              npcs = 2000, approx = FALSE,
              verbose = FALSE)
```



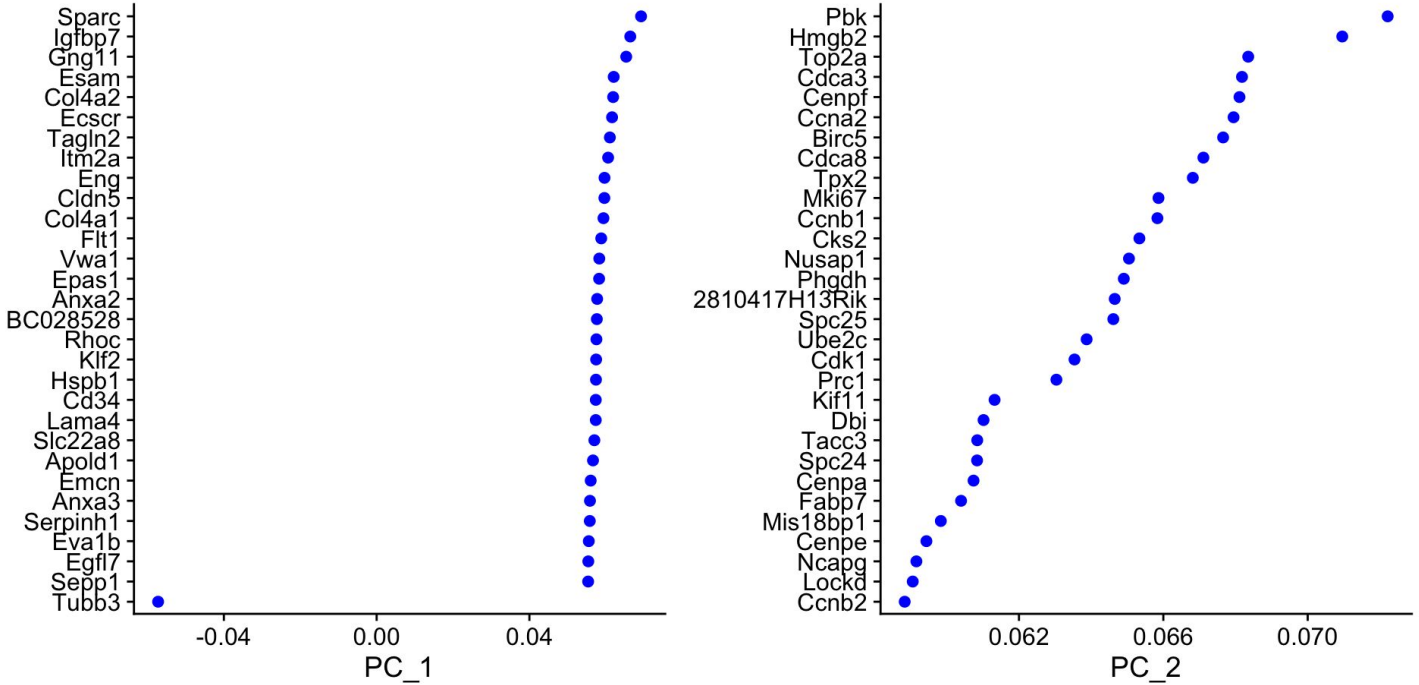
PCA

```
ElbowPlot(obj, ndims = 30)
```



PCA

```
VizDimLoadings(obj, dims = 1:2, reduction = "pca")
```



Clustering

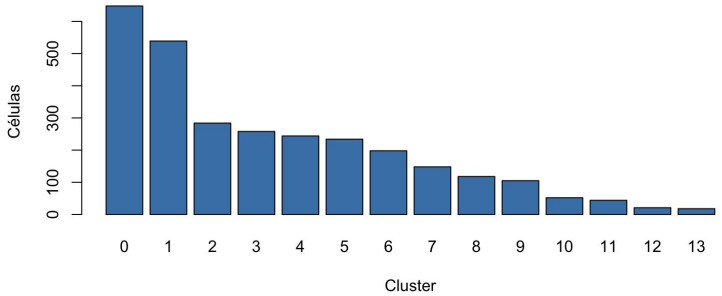
```
obj <- FindNeighbors(obj, dims = 1:20, verbose = FALSE)
obj <- FindClusters(obj, resolution = 0.5, random.seed = 42, verbose = FALSE)
head(Idents(obj), 5)
nlevels(Idents(obj)) # Número de clusters
```

```
126_GGCGACTTCTTGCATT-126   64_AAGTCTGTCCTCATTA-64   10_ACACCAACACGGTTTA-10 104_TGCCAAAGTGCCTGTG-104
                       0                       6                       9                       2
  60 TACGGGCGTCGGGTCT-60
                       0
Levels: 0 1 2 3 4 5 6 7 8 9 10 11 12 13
[1] 14
```

```
tamano_clusters <- table(Idents(obj))
tamano_clusters
```

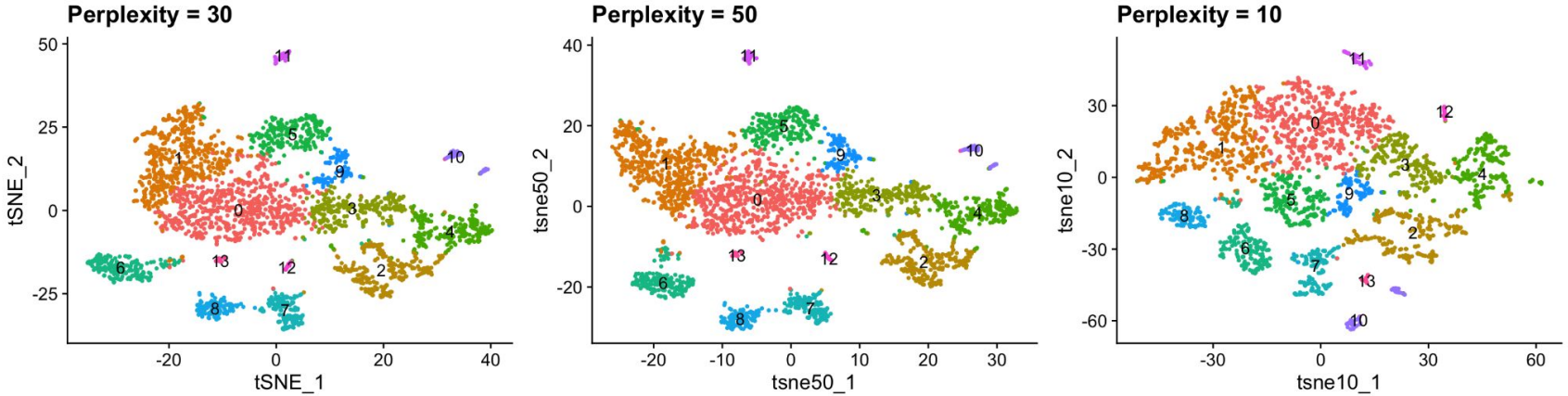
```
  0   1   2   3   4   5   6   7   8   9
 10  11  12  13
648 539 284 258 244 234 198 148 118 105
 52  44  21  18
```

```
barplot(tamano_clusters, col = "steelblue",  xlab = "Cluster", ylab
= "Células")
```



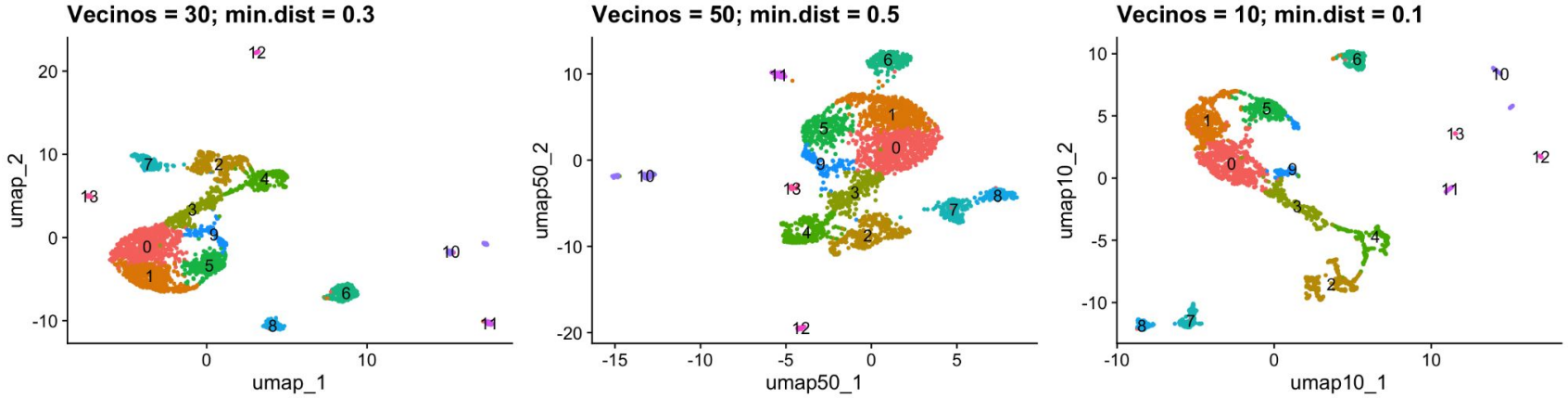
tSNE

```
obj <- RunTSNE(obj, dims = 1:20, perplexity = 30, seed.use = 42, verbose = FALSE)
obj <- RunTSNE(obj, dims = 1:20, perplexity = 50, seed.use = 42, verbose = FALSE, reduction.name = "tsne50")
obj <- RunTSNE(obj, dims = 1:20, perplexity = 10, seed.use = 42, verbose = FALSE, reduction.name = "tsne10")
plot1 <- DimPlot(obj, reduction = "tsne", label = TRUE) + ggtitle("Perplexity = 30") + NoLegend()
plot2 <- DimPlot(obj, reduction = "tsne50", label = TRUE) + ggtitle("Perplexity = 50") + NoLegend()
plot3 <- DimPlot(obj, reduction = "tsne10", label = TRUE) + ggtitle("Perplexity = 10") + NoLegend()
plot1 + plot2 + plot3
```



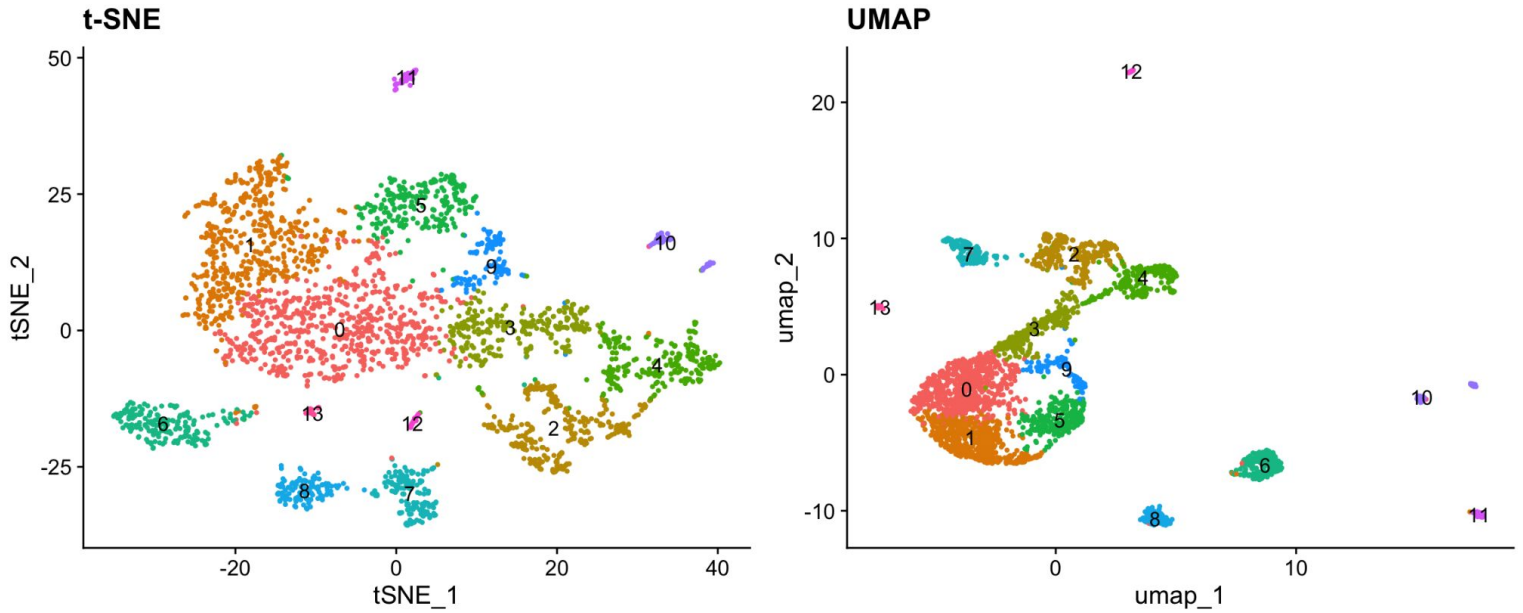
UMAP

```
obj <- RunUMAP(obj, dims = 1:20, n.neighbors = 30, min.dist = 0.3, seed.use = 42, verbose = FALSE)
obj <- RunUMAP(obj, dims = 1:20, n.neighbors = 50, min.dist = 0.5, seed.use = 42, verbose = FALSE, reduction.name
= "umap50")
obj <- RunUMAP(obj, dims = 1:20, n.neighbors = 10, min.dist = 0.1, seed.use = 42, verbose = FALSE, reduction.name
= "umap10")
plot1 <- DimPlot(obj, reduction = "umap", label = TRUE) + ggtitle("Vecinos = 30; min.dist = 0.3") + NoLegend()
plot2 <- DimPlot(obj, reduction = "umap50", label = TRUE) + ggtitle("Vecinos = 50; min.dist = 0.5") + NoLegend()
plot3 <- DimPlot(obj, reduction = "umap10", label = TRUE) + ggtitle("Vecinos = 10; min.dist = 0.1") + NoLegend()
plot1 + plot2 + plot3
```



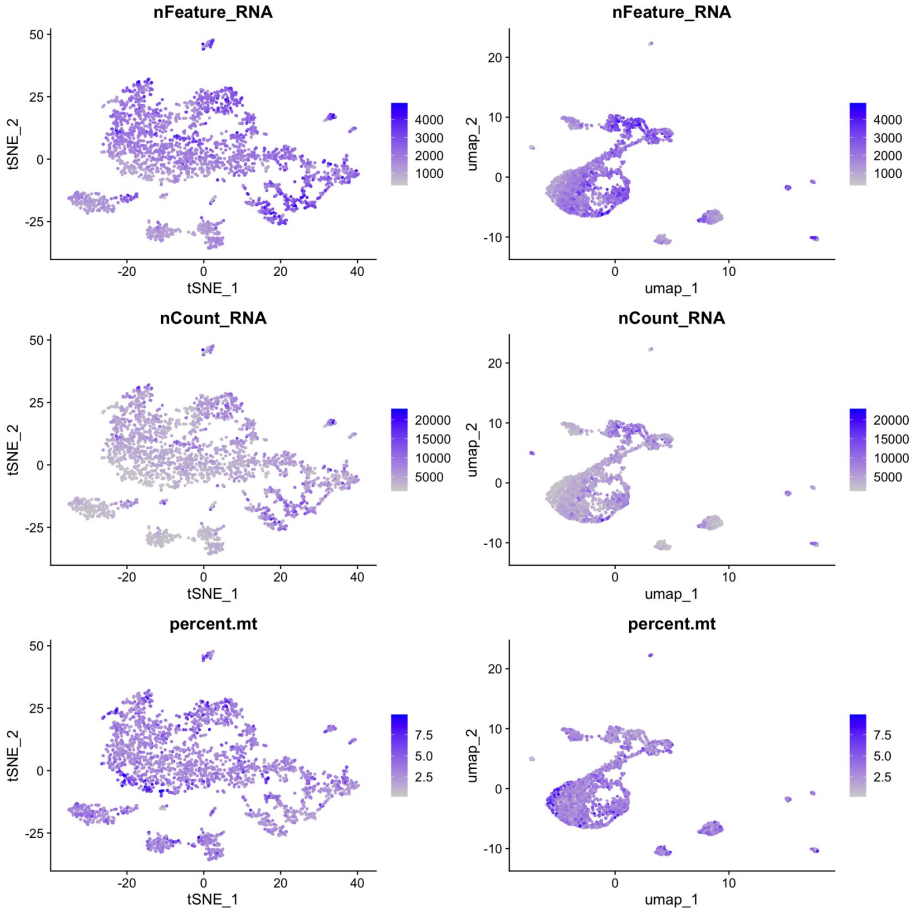
Comparación

```
plot1 <- DimPlot(obj, reduction = "tsne", label = TRUE) + ggtitle("t-SNE") + NoLegend()
plot2 <- DimPlot(obj, reduction = "umap", label = TRUE) + ggtitle("UMAP") + NoLegend()
plot1 + plot2
```



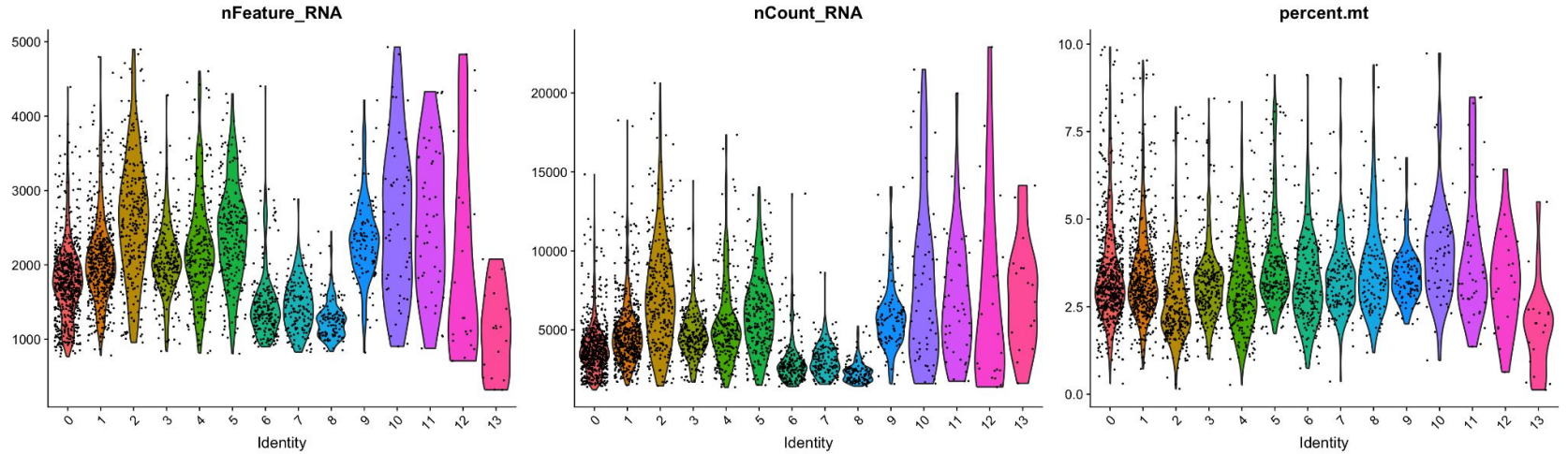
Comparación

```
plot1 <- FeaturePlot(obj, features = metricas_qc,
                     reduction = "tsne", ncol = 1)
plot2 <- FeaturePlot(obj, features = metricas_qc,
                     reduction = "umap", ncol = 1)
plot1 | plot2
```



Comparación

```
VlnPlot(obj, features = metricas_qc, layer = "counts", ncol = 3, pt.size = 0.1) + NoLegend()
```



FACULTAD DE MEDICINA
UNIVERSIDAD DE CHILE

Magíster en Informática Médica

Single-cell RNA-seq I

Noemí Gómez
29 de octubre, 2026

noemi.gomez@postgrado.uoh.cl